

DrugNet

Faster Drug Discovery Using Generative Deep Learning

Ali Kılıç

alikelic.2004@gmail.com

A technical record of an integrated computational drug-discovery system combining a SELFIES molecular autoencoder, Bayesian latent-density estimation, an adaptive population-based search algorithm, and attention-based drug–target affinity prediction.

Provenance. DrugNet (originally *İlaçNet: Derin Öğrenmeyle Hızlı İlaç Üretimi*) was designed, implemented and evaluated as an independent research project during 2020–2021, and placed in the top five of its category at Teknofest. This document is a retrospective technical write-up prepared in 2026 from the original project report and the original implementation notebook. It describes the system as it was built and evaluated at that time; no new experiments were run for this write-up, and no literature published after the project is cited or compared against. Every architectural detail is marked as either verified from the implementation or reported by the project; the two are never merged.

Revised technical edition • September 2026

Abstract

Early-stage drug discovery is constrained less by the size of chemical space than by the cost of deciding which region of it to examine. DrugNet was built to make that decision cheaply and repeatedly: it couples a generative molecular representation to a learned target-affinity model so that a search algorithm can propose molecules and score them against a chosen protein without leaving the computer.

The system has four parts. A Molecular Autoencoder (MAE) maps 128-token SELFIES sequences through an 85×84 embedding and two bidirectional GRU layers of 240 units per direction into an 84-dimensional continuous latent code, and reconstructs them with a two-layer 480-unit GRU decoder emitting an 85-way categorical distribution at each position. Because the autoencoder is deterministic rather than variational, a 12-component full-covariance Bayesian Gaussian Mixture Model (BGMM) is fitted afterwards to the encoded training set, supplying the sampling distribution the autoencoder does not itself provide. Advanced Evolutionary Spread Search (AESS), a modification of Virus Spread Optimization, searches that latent space with 400 carriers whose update rules depend on a carrier class, and adjusts its own exploration pressure through a logistic function of how long the incumbent best solution has survived. DeepADTP predicts drug-target interaction strength from 170-dimensional representations using causal dilated convolutions with dilation rates 1, 2, 4, 8, 16, bidirectional dot-product attention between the molecule and protein streams, and three 850-unit Mish layers. In the integrated loop AESS proposes latent vectors, the decoder turns them into molecules, and DeepADTP scores each one against the SARS-CoV-2 3CL protease; the prediction is negated because AESS minimises.

The project reports strong computational results. Under the MOSES benchmark metrics the MAE reached validity 1.0000, uniqueness@10k 0.9957, novelty 0.9560 and internal diversity 0.8898. DeepADTP reached MSE 0.121 and concordance index 0.890 on the project’s KIBA evaluation, ahead of the SimBoost, DeepDTA and AttentionDTA figures it was compared against. Fifteen runs of thirty iterations retained 308 candidate molecules whose mean predicted fitness was 14.9547 against 11.2692 for the comparison COVID-19 molecule set. On the Penalized LogP optimization benchmark AESS reached 53.02, against 44.87 for the same framework driven by unmodified VSO and 31.40 for the strongest external figure in the comparison table.

These are in-silico outcomes. The 308 retained molecules score highly under a learned surrogate objective; nothing in the project measures binding, inhibition, activity or safety. This document states each result at the strength the evidence supports, records the implementation exactly as it was written — including the defects a re-implementation would need to repair — and identifies where the search dynamics themselves qualify how a headline number should be read.

Keywords: drug discovery; generative deep learning; molecular autoencoder; SELFIES; latent-space optimization; metaheuristics; drug-target affinity; attention; COVID-19.

Contents

Abstract	2
Notation and abbreviations	6
1 Introduction	7
1.1 Speed as a constraint on drug discovery	7
1.2 Why a learned representation helps	7
1.3 Connecting generation to target scoring	7
1.4 Aim, questions and contributions	8
2 Background and related work	9
2.1 Molecular representations: SMILES and SELFIES	9
2.2 Autoencoders and latent molecular optimisation	9
2.3 Prior molecular-generation approaches	9
2.4 Drug–target affinity prediction	10
2.5 Metaheuristic search and the exploration–exploitation trade-off	10
3 System overview	10
4 Data and preprocessing	12
4.1 Molecular corpus	12
4.2 Drug–target data	12
4.3 Tensor conventions	12
5 Molecular Autoencoder	13
5.1 Role in the system	13
5.2 Encoder	13
5.3 Decoder	15
5.4 Training objective	16
5.4.1 What the implementation actually optimises	16
5.5 Training configuration	17
6 Latent density modelling with a Bayesian Gaussian mixture	17
6.1 Why a second model is needed at all	17
6.2 The mixture model	18
6.3 Priors, responsibilities, and variational fitting	19
6.4 Sampling	19
6.5 Configuration and what the implementation fixes	20
7 Advanced Evolutionary Spread Search	20
7.1 The algorithm as a population process	20
7.2 Assignment and selection	21
7.3 Mutation	21
7.4 Infection	22
7.5 Recovery	23
7.6 Adaptation	23
7.7 The redesigned healthy-carrier update	24

7.8	Hyperparameters	26
8	DeepADTP: attention-based drug–target prediction	26
8.1	Objective	26
8.2	Dilated convolution and the receptive field	26
8.3	Residual blocks	27
8.4	Bidirectional attention	28
8.5	Regression head and objective	28
9	The integrated pipeline	29
9.1	Training, then search	29
9.2	One optimisation iteration	29
9.3	Why the surrogate structure matters	30
10	Experimental protocol	30
10.1	Generative evaluation of the autoencoder	30
10.2	Affinity evaluation	31
10.3	Integrated target search	31
10.4	Property optimisation and the VSO ablation	31
11	Results	32
11.1	Molecular generation	32
11.2	Drug–target affinity	32
11.3	Integrated target search	33
11.4	Property optimisation and the search ablation	34
12	Discussion	35
12.1	What the generation metrics say about the representation	35
12.2	The value of separating the density model	35
12.3	Adaptation as feedback control	35
12.4	Surrogate objectives and what they license	36
12.5	The system as a whole	36
13	Limitations	36
13.1	Computational prediction is not experimental evidence	36
13.2	What the search dynamics imply for the Penalized LogP result	37
13.3	Interpretation of the MOSES comparison	37
13.4	Statistical uncertainty	37
13.5	The representational ceiling	38
13.6	Reproducibility of the supplied implementation	38
14	Future work	38
15	Conclusion	38
	References	40
A	Consolidated hyperparameters	43

B	Bayesian Gaussian mixture: variational updates	45
B.1	Complete-data likelihood	45
B.2	Variational factorisation and the updates	45
B.3	Effective components and posterior sampling	46
C	Notebook audit and reproducibility caveats	46
C.1	Defects that prevent execution	47
C.2	Defects that change behaviour silently	47
C.3	Cosmetic evidence of editing	48
C.4	Recommended replication policy	48
D	Source fidelity	48

Notation and abbreviations

Scalars are italic (a , γ), vectors bold lowercase ($\mathbf{z}$, $\mathbf{u}$), and matrices bold uppercase ($\mathbf{W}$, $\mathbf{\Sigma}$). The symbol $\odot$ denotes the elementwise (Hadamard) product, and subscripted indices such as u_j refer to coordinate j of the corresponding vector. Random draws are written $u_j \sim \text{Uniform}[0, 2)$, meaning each coordinate is drawn independently.

Table 1: Symbols used throughout the document. Where the original project report and the implementation use different names for the same quantity, the implementation name is given in typewriter font.

Symbol	Meaning
$\mathbf{x} \in \{0, \dots, 84\}^{128}$	Molecule as a padded integer SELFIES token sequence
$\mathbf{P} \in \{0, \dots, 20\}^{1500}$	Protein as a padded integer amino-acid sequence
$\mathbf{z} \in \mathbb{R}^{84}$	Latent molecular code
E_ϕ	MAE encoder, $E_\phi : \mathbf{x} \mapsto \mathbf{z}$
D_θ	MAE decoder, $D_\theta : \mathbf{z} \mapsto [0, 1]^{128 \times 85}$
g_ω	DeepADTP affinity model, $g_\omega(\mathbf{P}, \mathbf{x}) \in \mathbb{R}$
$p_\psi(\mathbf{z})$	Fitted BGMM density over latent codes
$K = 12$	Number of mixture components
π_k, μ_k, Σ_k	Mixture weight, mean and covariance of component k
$\Lambda_k = \Sigma_k^{-1}$	Component precision matrix
$c_n \in \{1, \dots, K\}$	Latent component assignment of encoded molecule n
r_{nk}	Responsibility of component k for molecule n
$\mathbf{z}^*$	Latent vector of the critical (incumbent best) carrier
$q \in \{\text{H}, \text{M}, \text{S}, \text{C}\}$	Carrier class: healthy, mild, severe, critical
a	Critical-carrier age, in selection cycles <code>crit_age</code>
$p(a)$	Logistic adaptation coefficient
γ	Mild attraction coefficient <code>y_exploit</code>
b	Severe intensity decay coefficient <code>decay</code>
H	Maximum healthy carriers contacted by one infected carrier <code>H</code>
ρ_q	Infection probability of an infector of class q <code>infection_rates</code>
$f(\mathbf{z})$	AESS fitness, minimised
u_t, r_t	GRU update and reset gates

Table 2: Abbreviations.

AAE	Adversarial Autoencoder	GRU	Gated Recurrent Unit
AE	Autoencoder	MAE	Molecular Autoencoder
AESS	Advanced Evolutionary Spread Search	MSE	Mean Squared Error
BGMM	Bayesian Gaussian Mixture Model	QED	Quantitative Estimate of Drug-likeness
CI	Concordance Index	RL	Reinforcement Learning
CNN	Convolutional Neural Network	SELFIES	Self-Referencing Embedded Strings
DTA	Drug-Target Affinity	SMILES	Simplified Molecular Input Line Entry System
ELBO	Evidence Lower Bound	SA	Synthetic Accessibility
GAN	Generative Adversarial Network	VAE	Variational Autoencoder
GMM	Gaussian Mixture Model	VSO	Virus Spread Optimization

1 Introduction

1.1 Speed as a constraint on drug discovery

The COVID-19 pandemic made the cost of a slow therapeutic pipeline unusually legible. The original project report records that by 25 January 2021 the disease had caused approximately 25,000 deaths in Türkiye and 2.13 million worldwide [1], and argues that population growth and denser transport networks make the spread of a future outbreak faster still [2]. Against that background, compressing the earliest stage of drug discovery is not only a question of research budget; it changes how quickly candidate interventions can enter laboratory and clinical evaluation at all.

Conventional discovery proceeds through a long sequence of decisions. A biologically relevant target must be identified, a mechanism of modulation established, molecular structures with suitable interaction patterns found, potency and selectivity assessed, and chemical and pharmacological properties optimised — and only then does the work enter the increasingly expensive experimental stages. The source report characterises the whole pipeline as roughly a twelve-year, billion-dollar undertaking [3]. DrugNet addresses one narrow part of it: the computational generation and prioritisation of candidate molecules before any experimental work begins.

The difficulty at that stage is one of scale. Estimates of the drug-like chemical space run to 10^{23} molecules and beyond, which forecloses enumeration as a strategy. A computational system that hopes to search it usefully therefore needs two things that are easy to state and hard to build: a representation in which chemically meaningful candidates can be generated and smoothly modified, and an objective that ranks those candidates for a particular biological target. DrugNet supplies the first with the MAE, the BGMM and AESS, and the second with DeepADTP.

1.2 Why a learned representation helps

Neural networks are flexible function approximators fitted from data. In generative modelling the task is harder than mapping inputs to labels: the model must capture enough of a data distribution to produce new samples that belong to the domain while differing from any individual training example. Two families motivated the original project — generative adversarial networks [4], which learn through competition between a generator and a discriminator, and autoencoders [5], which learn an encoder that compresses and a decoder that reconstructs. For molecular design the autoencoder formulation has a specific advantage: its latent vector is a continuous space that an optimisation algorithm can search.

That continuity is the entire point. Molecular strings are discrete, and altering one token of a SMILES string can change the molecule substantially or make it syntactically invalid. A latent representation replaces the combinatorial object with a vector of real numbers, and if nearby latent vectors tend to decode into related molecules, a numerical optimiser can work in the smooth space instead of editing strings. Every design decision in DrugNet follows from taking that idea seriously — including the ones, discussed in [Section 13](#), where the assumption of local smoothness is pushed further than it can be relied upon.

1.3 Connecting generation to target scoring

Most of the literature the project drew on treats molecular generation and drug–target affinity prediction as separate problems. On the generation side it cites Junction Tree

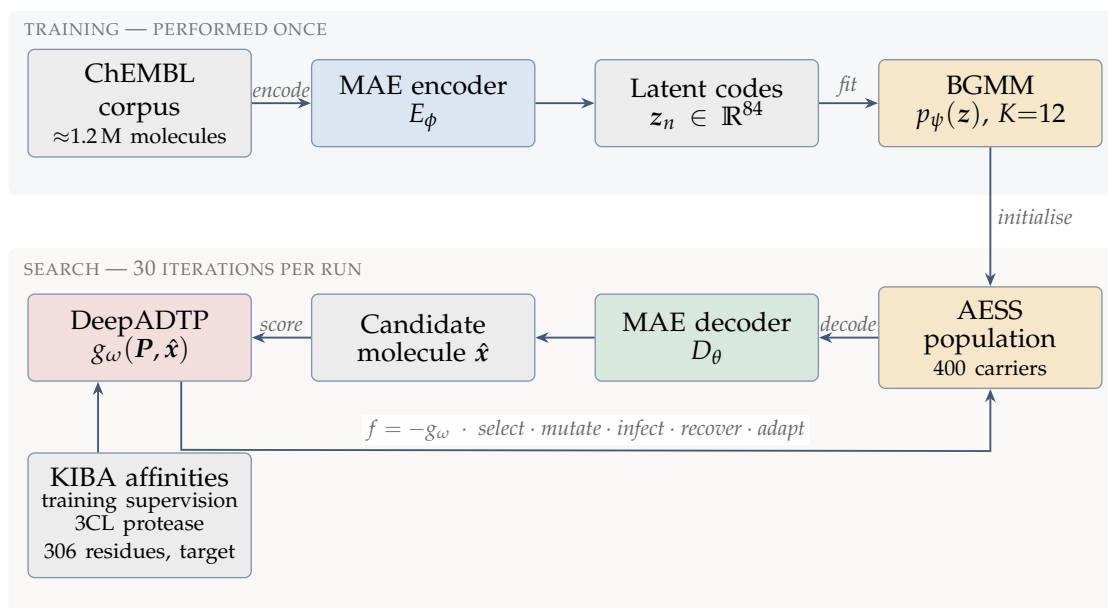


Figure 1: The DrugNet search loop. Training (upper path) fits the autoencoder on the ChEMBL-derived corpus, encodes that corpus, and fits a BGMM to the resulting latent codes; DeepADTP is trained separately on KIBA. Search (lower cycle) draws an initial carrier population from the BGMM, and thereafter repeats: decode each carrier’s latent vector to a molecule, score it against the target protein with DeepADTP, negate the score to obtain a minimisation fitness, and update the carrier population. The autoencoder is the only component that crosses between discrete chemistry and continuous search.

VAE [6], LatentGAN [7], Molecular Swarm Optimization [8], GEGL [9], GCPN [10] and MolDQN [11]; on the prediction side, DeepDTA [12] and AttentionDTA [13].

DrugNet’s architectural claim is that the two should be joined. A generator on its own does not know which biological target matters, and an affinity predictor on its own cannot search chemical space. Using DeepADTP as the surrogate objective inside AESS closes the loop, so that generation is steered towards molecules the target model scores highly. Figure 1 shows the resulting system.

1.4 Aim, questions and contributions

The aim of the project was to build a computational framework that shortens and simplifies early-stage candidate discovery by combining deep learning with metaheuristic optimisation. It is best described as experimental development rather than theory: it assembles components, modifies a prior search algorithm, evaluates the result against published benchmark figures, and asks whether the assembled system produces candidates with better predicted properties. The success criterion is comparative throughout.

Five questions organise the work. Can a deterministic molecular autoencoder learn a compact continuous representation that both reconstructs and supports diverse generation? Can a BGMM approximate the empirical latent distribution well enough to initialise and sustain a search, without forcing a prior on the autoencoder during training? Can an adaptive modification of Virus Spread Optimization improve latent-space exploration and exploitation? Can a dilated-convolution and attention model predict drug–target affinity accurately enough to serve as a surrogate fitness function? And when the parts

are connected, does the system find molecules with higher model-predicted target fitness than a comparison set, and higher property-optimisation scores than the selected prior methods?

The corresponding contributions are a SELFIES-based molecular autoencoder with bidirectional GRU encoding, layer normalisation and latent-magnitude regularisation; a BGMM fitted after training to model the empirical latent density while leaving the autoencoder objective deterministic; AESS, which extends VSO with an adaptation mechanism driven by critical-carrier age and a redesigned healthy-carrier update; DeepADTP, an affinity model built from dilated convolutions, residual connections, two directional attention modules and Mish nonlinearities; and the integrated loop in which decoded candidates are scored against a target protein and the score is fed back to the latent optimiser.

2 Background and related work

2.1 Molecular representations: SMILES and SELFIES

SMILES [14] encodes molecular structure as a character string. It is compact and ubiquitous, which is why early sequence models of molecules adopted it. Its weakness is that its grammar is not closed under modification: an arbitrary edit can produce a string that parses to nothing, or to a chemically impossible structure. For a search algorithm this is expensive in a specific way — steps spent proposing unparseable strings yield no gradient of information about the objective, so the effective search budget shrinks.

SELFIES [15] was designed to remove this failure mode. Its tokens carry derivation rules that constrain how a sequence is interpreted, so that essentially any token string decodes to a syntactically valid molecule. DrugNet converts its entire corpus to SELFIES for exactly this reason, and the SELFIES tokens become the discrete vocabulary of both neural models. The validity figure of 1.0000 reported in [Section 11](#) is a direct consequence of this representational choice rather than an independent property of the learned model, and [Section 12](#) treats it accordingly.

2.2 Autoencoders and latent molecular optimisation

An autoencoder learns an encoder E mapping a molecule x to a latent vector $z = E(x)$ and a decoder D mapping z back towards x . For optimisation, z is the object of interest: once a useful latent geometry exists, a search algorithm can perturb z , decode, and score the result. MSO [8] follows the same strategy, and is the closest prior work to DrugNet in that both separate representation learning from the search that exploits it.

The MAE is deterministic rather than variational. A variational autoencoder [16] regularises its latent codes towards a chosen prior during training, which yields a distribution to sample from but constrains the representation. Following the reasoning behind deterministic regularised autoencoders [17], the MAE instead constrains only the magnitude of the latent codes and defers the question of a sampling distribution to a separate model fitted afterwards. [Section 6](#) takes up what is gained and what is paid for that choice.

2.3 Prior molecular-generation approaches

Junction Tree VAE [6] generates molecules by assembling chemically meaningful substructures, which removes the validity problem structurally rather than representationally,

and the original report singles it out for that reason. LatentGAN [7] pairs latent representations with adversarial generation. GCPN [10] and MolDQN [11] formulate molecular graph construction as a reinforcement-learning problem, optimising properties during generation rather than afterwards. GEGL [9] combines genetic exploration with a learned generator.

These methods differ along three axes: the representation being searched, whether validity is enforced by the representation or by the generation mechanism, and whether property optimisation happens during generation or in a separate search. DrugNet sits firmly in the last category. The generator establishes a continuous space; AESS performs the goal-directed search over it. That separation is what makes the surrogate objective swappable, and is the source of most of the system’s modularity.

2.4 Drug–target affinity prediction

A molecule is useful only if it interacts with the intended biological system. Computational affinity prediction estimates that interaction from molecular and protein representations alone. DeepDTA [12], an early deep model in this area, processes a SMILES string and an amino-acid sequence with separate convolutional stacks before regressing an affinity value. AttentionDTA [13] adds attention, letting the model weight cross-sequence features rather than relying on pooled summaries.

DeepADTP keeps that objective but changes both the feature extractor and the interaction mechanism. Dilated convolutions [18] widen the receptive field geometrically with depth without a recurrent pass over a 1500-position protein sequence; residual connections [19] let earlier features reach later layers directly; and two directional attention modules [20] allow molecular features to condition on protein features and the reverse, before regression.

2.5 Metaheuristic search and the exploration–exploitation trade-off

Any population-based search must balance two pressures. Exploration moves the population into under-sampled regions and guards against premature convergence; exploitation concentrates effort near solutions that already look strong and is what produces local refinement. Virus Spread Optimization [21] encodes this tension in the structure of its population: carriers occupy classes whose update rules, borrowed metaphorically from viral transmission, differ in step size and in how strongly they are attracted to the incumbent best solution.

VSO fixes that balance for the duration of a run. The central modification in AESS is to make it a function of the search’s own progress, so that exploration pressure rises when the incumbent stops improving. Section 7 develops this in full.

3 System overview

Before the components are described individually it is worth stating the pipeline as a single composition of maps, because each component exists to supply something the previous one lacks.

A molecule is encoded,

$$z = E_{\phi}(x), \quad E_{\phi} : \{0, \dots, 84\}^{128} \rightarrow \mathbb{R}^{84}, \quad (1)$$

producing a continuous code in place of a discrete string. Encoding the whole training corpus gives an empirical set of codes $Z = \{z_n\}_{n=1}^N$, whose density is approximated by a

Table 3: The four components, the problem each solves, and where its output is consumed. Every component supplies a capability that the one before it lacks, which is why none of them is useful in isolation for the task the system as a whole performs.

Component	Problem addressed	Output consumed by
MAE encoder	Compress discrete molecules into a searchable continuous representation	BGMM fitting; latent analysis
BGMM	Recover a sampling distribution the deterministic autoencoder does not define	AESS initialisation and recovery
AESS	Search for improved latent vectors under a shifting exploration/exploitation balance	MAE decoder
MAE decoder	Convert proposed latent vectors back into molecular sequences	DeepADTP
DeepADTP	Estimate molecule–target interaction strength cheaply enough to call thousands of times	AESS fitness function

fitted mixture,

$$p_{\psi}(\mathbf{z}) = \sum_{k=1}^{12} \pi_k \mathcal{N}(\mathbf{z} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \approx p_{\text{latent}}(\mathbf{z}). \quad (2)$$

This is the step that recovers a sampling distribution the deterministic autoencoder never committed to. AESS maintains a population of candidate codes $\mathbf{z}_t^{(1)}, \dots, \mathbf{z}_t^{(400)}$, initialised from p_{ψ} and updated by class-dependent stochastic rules. Each candidate is decoded and discretised,

$$\hat{\mathbf{x}} = \arg \max_j D_{\theta}(\mathbf{z})_{t,j}, \quad D_{\theta} : \mathbb{R}^{84} \rightarrow [0, 1]^{128 \times 85}, \quad (3)$$

and scored against a target protein $\mathbf{P}$,

$$s = g_{\omega}(\mathbf{P}, \hat{\mathbf{x}}) \in \mathbb{R}. \quad (4)$$

Because AESS minimises, the target-affinity objective enters with its sign reversed,

$$f_{\text{3CL}}(\mathbf{z}) = -g_{\omega}(\mathbf{P}_{\text{3CL}}, \arg \max D_{\theta}(\mathbf{z})). \quad (5)$$

Table 3 summarises what each component contributes and what consumes its output. The structure is deliberately linear at training time and cyclic at search time: nothing in the loop requires retraining, so a single trained system supports arbitrarily many searches against different targets or different objectives.

Two facts about this composition matter for interpreting everything that follows. First, the decoder is applied to latent vectors that were never produced by encoding a real molecule, so the quality of the candidates depends on how the decoder behaves away from its training codes — a property the training objective does not directly control. Second, the objective being optimised is a learned model, not a measurement, so the search optimises whatever that model rewards, including its errors. Both points are developed in Sections 12 and 13.

4 Data and preprocessing

4.1 Molecular corpus

The molecular corpus was assembled from ChEMBL [22]. Roughly 6,800 molecules listed under a COVID-19 heading were combined with approximately 1.9 million molecules from the main ChEMBL release, and filtering reduced the union to approximately 1.2 million molecules used to train the autoencoder.

Filtering had three purposes: removing molecules unlikely to behave as drugs, unifying the representation, and guaranteeing that every retained molecule could be written in the vocabulary shared by both neural models. Molecules from the main set violating any Lipinski rule [23] or scoring below 0.5 on the quantitative estimate of drug-likeness [24] were discarded. The report is explicit that the QED filter was *not* applied to the COVID-19 subset, on the grounds that those molecules had already been selected for relevance to the target disease and a generic drug-likeness threshold would have discarded exactly the structures of interest. Retained molecules were converted from SMILES to SELFIES, and those whose SELFIES length fell outside [10, 128] tokens were removed.

4.2 Drug–target data

DeepADTP was trained on KIBA [25], which pairs molecular structures with protein sequences and an interaction score. Each KIBA molecule was converted to SELFIES, and the resulting token set was adopted as the project’s master molecular vocabulary. Any molecule in the autoencoder corpus containing a token outside that vocabulary was removed, so that a single integer encoding serves both the generative and the predictive model — without this, a molecule proposed by the decoder could contain a token DeepADTP had never been trained to embed, and the search loop would break at its most important join.

The original report describes the KIBA interaction labels as K_i values. That characterisation is preserved here without substitution, since the supplied material does not contain the preprocessing code that would settle the point.

4.3 Tensor conventions

The supplied notebook begins from preprocessed arrays and therefore does not reproduce the ChEMBL and KIBA cleaning pipeline. It does, however, fix the conventions all downstream shapes depend on:

$$\mathbf{x} \in \{0, \dots, 84\}^{128}, \quad \mathbf{P} \in \{0, \dots, 20\}^{1500}, \quad \mathbf{z} \in \mathbb{R}^{84}. \quad (6)$$

The token table in the notebook holds exactly 85 entries. Index 0 is the empty string, used as the padding symbol; indices 1 through 84 are SELFIES tokens, running from common atoms and bonds (`[C]`, `[=O]`, `[nHexpl]`) through branch and ring operators (`[Branch1_1]`, `[Expl=Ring2]`) to rarer species (`[Zn+2expl]`, `[P@@expl]`). Molecules shorter than 128 tokens are right-padded with zeros. The protein vocabulary has 21 symbols, consistent with 20 amino acids plus padding, and protein sequences are padded to a fixed length of 1500.

The SARS-CoV-2 3CL protease sequence embedded in the notebook contains 306 encoded residues, padded with zeros from position 306 to 1499. This length is an independent consistency check that the sequence is what it claims to be: the SARS-CoV-2

Table 4: The preprocessing pipeline as reconstructed from the report and the notebook’s tensor conventions. Steps 1–5 are described in the report only; steps 6–11 are additionally confirmed by the array shapes and vocabulary table in the notebook.

#	Operation	Purpose
1	Collect ChEMBL main set (≈ 1.9 M) and the COVID-19 subset ($\approx 6,800$)	Build a large, partly disease-relevant corpus
2	Filter the main set by the Lipinski rules and $\text{QED} \geq 0.5$	Bias the corpus towards drug-like structures
3	Convert SMILES to SELFIES	Make token sequences robust to invalid generation
4	Keep SELFIES lengths in $[10, 128]$	Bound sequence length and training cost
5	Load KIBA molecule–protein–affinity triples	Construct the affinity training set
6	Define the 84-token molecular vocabulary from KIBA	Unify tokenisation across MAE and DeepADTP
7	Remove corpus molecules with out-of-vocabulary tokens	Guarantee every molecule is representable by both models
8	Map tokens to integers 1–84, reserving 0 for padding	Produce network inputs
9	Right-pad molecular sequences to length 128	Enable fixed-shape batched training
10	Tokenise proteins with the separate 21-symbol vocabulary, pad to 1500	Produce protein inputs
11	Align molecule, protein and affinity rows	Prepare supervised DeepADTP examples

main protease is 306 amino acids long. It also means that for this particular target, 1194 of the 1500 protein positions carry padding — a point taken up in [Section 8](#).

5 Molecular Autoencoder

5.1 Role in the system

The MAE is the only component that crosses between discrete chemistry and continuous search. Its encoder compresses a molecular sequence into a vector; its decoder maps vectors back to sequences. Everything AESS does happens in the space this pair defines, and everything DeepADTP scores comes out of the decoder. [Figure 2](#) shows the architecture as implemented.

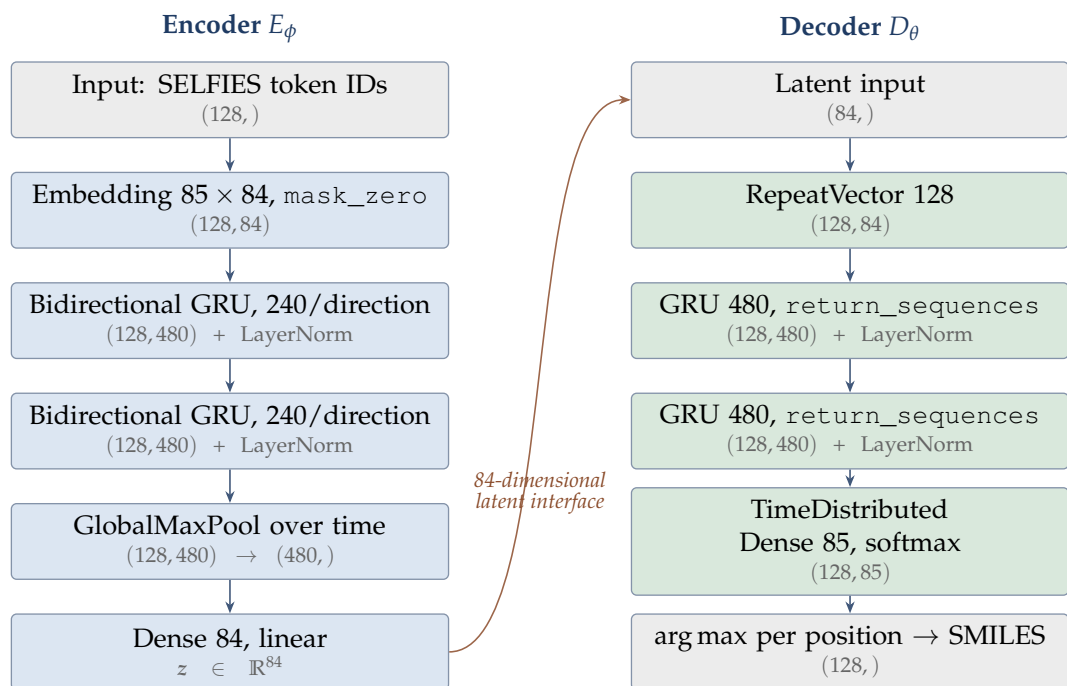
5.2 Encoder

Let $x_t \in \{0, \dots, 84\}$ be the token at position $t \in \{1, \dots, 128\}$ and let $W_{\text{emb}} \in \mathbb{R}^{85 \times 84}$ be the learned embedding matrix. Each token is mapped to a dense vector

$$e_t = W_{\text{emb}}^\top \mathbf{1}_{x_t} \in \mathbb{R}^{84}, \quad (7)$$

where $\mathbf{1}_{x_t}$ is the one-hot indicator of token x_t . Embedding is a lookup, so [Equation \(7\)](#) is a notational convenience rather than a matrix product performed at runtime; writing it this way makes the parameter count explicit ($85 \times 84 = 7,140$).

The embedded sequence is processed by gated recurrent units [26]. A GRU maintains a hidden state h_t and two gates. To avoid a clash with the latent code z , the update gate



Encoder parameters 1,558,584 • decoder parameters 2,243,125 • vocabulary IDs 0–84 with 0 reserved for padding

Figure 2: The Molecular Autoencoder, with shapes and parameter counts taken from the implementation. The encoder is a two-layer bidirectional GRU stack reduced to a fixed-length code by max-pooling over time; the decoder broadcasts the code across 128 positions and emits an independent categorical distribution at each. Note the asymmetry: the encoder sees the token sequence, whereas the decoder sees only the 84-dimensional code, repeated. All sequence structure in a reconstruction must therefore be reproduced by the decoder’s recurrence from a single vector.

is written u_t and the reset gate r_t :

$$u_t = \sigma(W_u e_t + U_u h_{t-1} + b_u), \quad (8)$$

$$r_t = \sigma(W_r e_t + U_r h_{t-1} + b_r), \quad (9)$$

$$\tilde{h}_t = \tanh(W_h e_t + U_h (r_t \odot h_{t-1}) + b_h), \quad (10)$$

$$h_t = (1 - u_t) \odot h_{t-1} + u_t \odot \tilde{h}_t. \quad (11)$$

Here σ is the logistic function applied elementwise, $W_\bullet$ and $U_\bullet$ are learned weight matrices and $b_\bullet$ learned biases.

The two gates do different jobs, and the distinction matters for what the encoder can represent. The reset gate r_t in Equation (10) controls how much of the previous state enters the *candidate* state: driving r_t towards zero lets the unit propose a new representation that ignores its history, which is what allows a molecule’s structure to be re-anchored at a ring closure or a branch token. The update gate u_t in Equation (11) then interpolates between keeping the old state and adopting the candidate. Because the interpolation is a convex combination, information can survive many steps unchanged when u_t is small — the mechanism that lets a token at position 3 still influence the representation at position 90.

Remark 5.1. The equations in the original report use r_t for the quantity that interpolates the hidden state and g_t for the quantity that gates the candidate, which inverts the usual naming. The recurrences themselves are the same as [Equations \(8\) to \(11\)](#); only the labels differ, and this document uses the conventional assignment throughout.

The encoder runs two bidirectional GRU layers, each with 240 units per direction. Writing $\vec{h}_t$ and $\overleftarrow{h}_t$ for the forward and backward states, the layer output at position t is their concatenation,

$$h_t = \left[\vec{h}_t; \overleftarrow{h}_t \right] \in \mathbb{R}^{480}, \quad (12)$$

so that every position carries context from tokens on both sides of it. Layer normalisation [27] follows each bidirectional layer.

The variable-length sequence must become a fixed-length vector. The encoder does this by global max-pooling over time: writing $H \in \mathbb{R}^{128 \times 480}$ for the final contextual sequence,

$$m_c = \max_{1 \leq t \leq 128} H_{t,c}, \quad c = 1, \dots, 480, \quad (13)$$

followed by a linear projection to the latent code,

$$z = W_{\text{lat}} m + b_{\text{lat}} \in \mathbb{R}^{84}. \quad (14)$$

Max-pooling is a deliberate choice with a consequence worth naming. It asks, for each of the 480 channels, only *whether* a feature was detected anywhere in the sequence, discarding where it occurred and how often. This gives an encoder that is robust to padding and to positional shifts, but it also means position information reaches the code only through whatever the recurrent layers have already folded into the channel activations.

The encoder holds 1,558,584 trainable parameters.

5.3 Decoder

The decoder maps a code back to a distribution over sequences. It repeats z across all 128 positions, passes the result through two 480-unit GRU layers with layer normalisation, and applies a shared dense softmax head at every position. Writing $\ell_{t,j}(z)$ for the logit of token j at position t ,

$$p_\theta(x_t = j \mid z) = \frac{\exp(\ell_{t,j}(z))}{\sum_{m=0}^{84} \exp(\ell_{t,m}(z))}, \quad (15)$$

so the decoder defines

$$D_\theta : \mathbb{R}^{84} \rightarrow [0, 1]^{128 \times 85}, \quad p_\theta(x \mid z) = \prod_{t=1}^{128} p_\theta(x_t \mid z). \quad (16)$$

The factorisation in [Equation \(16\)](#) is what the training objective assumes: given the code, tokens are conditionally independent across positions. The decoder is not autoregressive — it never conditions position t on the token emitted at position $t - 1$ — so all coordination between positions must be carried by the recurrent state driven from the repeated code.

During search, a candidate molecule is recovered deterministically,

$$\hat{x}_t = \arg \max_{j \in \{0, \dots, 84\}} D_\theta(z)_{t,j}, \quad (17)$$

and the resulting token sequence is converted back to SMILES by the SELFIES decoder. The decoder holds 2,243,125 trainable parameters.

The decoder’s real job in DrugNet is not reconstruction. It is applied overwhelmingly to latent vectors that no encoder produced, because that is what a search algorithm generates. Its behaviour on those vectors — whether nearby codes decode to related molecules, and how far from the training codes it remains well-behaved — determines whether latent search means anything. Nothing in the training objective controls this directly, which is a structural limitation of the design rather than an implementation shortcoming; [Section 13](#) returns to it.

5.4 Training objective

Three regularisers are described in the report: a reconstruction term, a penalty on latent magnitude, and L_2 weight decay on the decoder.

For a minibatch B with sequence length $T = 128$, the reconstruction term is token-level categorical cross-entropy,

$$\mathcal{L}_{\text{rec}} = -\frac{1}{|B|} \sum_{n \in B} \sum_{t=1}^{128} \log p_{\theta}(x_{n,t} | E_{\phi}(\mathbf{x}_n)), \quad (18)$$

which is the negative log-likelihood of the observed sequence under [Equation \(16\)](#). Minimising it maximises the probability the decoder assigns to the correct token at every position.

The latent penalty constrains the magnitude of the code,

$$\mathcal{L}_{\text{lat}} = \frac{1}{4} \frac{1}{|B| \cdot 84} \sum_{n \in B} \|E_{\phi}(\mathbf{x}_n)\|_2^2 = \frac{1}{4} \text{mean}(\mathbf{z} \odot \mathbf{z}). \quad (19)$$

This is the mechanism that substitutes for a variational prior. It does not shape the latent distribution into any particular family; it only stops the codes from drifting to arbitrarily large magnitudes, which keeps the region a search algorithm has to cover bounded in practice. The coefficient $\frac{1}{4}$ is fixed in the implementation rather than tuned.

The decoder weight penalty is

$$\mathcal{L}_{\text{dec}} = \lambda \|\boldsymbol{\theta}_{\text{dec}}\|_2^2, \quad \lambda = 0.01, \quad (20)$$

declared on the kernels of both decoder GRUs. The report’s stated motivation is to push the decoder towards a broader account of the molecular domain rather than memorisation of individual training sequences — which matters here more than in a typical autoencoder, precisely because the decoder will be queried off-distribution.

The intended total objective is therefore $\mathcal{L}_{\text{MAE}} = \mathcal{L}_{\text{rec}} + \mathcal{L}_{\text{lat}} + \mathcal{L}_{\text{dec}}$.

5.4.1 What the implementation actually optimises

The implemented training step differs from the intended objective in one respect that a reproduction must know about. It computes

$$\mathcal{L}_{\text{code}}(\mathbf{x}) = 128 \cdot \text{SCE}(\mathbf{x}, D_{\theta}(E_{\phi}(\mathbf{x}))) + \frac{1}{4} \text{mean}(E_{\phi}(\mathbf{x}) \odot E_{\phi}(\mathbf{x})), \quad (21)$$

where SCE is sparse categorical cross-entropy under its default reduction, which averages over both the batch and the 128 sequence positions. Multiplying by 128 converts that time-average back into a per-sequence sum, so the first term of [Equation \(21\)](#) is exactly [Equation \(18\)](#).

Table 5: MAE configuration. Values marked *code* are read directly from the implementation; values marked *report* come from the project report and do not appear in the supplied notebook, which loads pretrained weights rather than containing a training call.

Item	Value	Source
Latent dimension	84	code
Token vocabulary	85 IDs: padding 0 + 84 SELFIES tokens	code
Sequence length	128	code
Embedding	85×84 , <code>mask_zero=True</code>	code
Encoder recurrence	$2 \times$ bidirectional GRU, 240 per direction ($\rightarrow 480$)	code
Pooling	global max over time	code
Decoder recurrence	$2 \times$ GRU, 480 units	code
Output head	time-distributed dense, 85-way softmax	code
Normalisation	layer normalisation after every recurrent block	code
Optimiser	Adam [28], $\beta_1 = 0.9$, $\beta_2 = 0.99$	code
Learning rate	5×10^{-4}	code
Gradient clipping	<code>clipvalue=0.5</code>	code
Declared decoder L_2	0.01 on both GRU kernels; <i>not applied</i> (Appendix C)	code
Epochs	50	report
Batch size	128	report
Encoder parameters	1,558,584	code
Decoder parameters	2,243,125	code

The L_2 term is a different matter. Both decoder GRUs declare `kernel_regularizer=12(0.01)`, but a custom training step must add the layer regularisation losses to the objective explicitly, and the implemented step does not. Under those semantics Equation (20) contributes nothing to the gradient. The reported method therefore includes a regulariser that the supplied code declares but does not apply. This is recorded as a reproducibility discrepancy in Appendix C rather than resolved by rewriting either source.

5.5 Training configuration

6 Latent density modelling with a Bayesian Gaussian mixture

6.1 Why a second model is needed at all

A variational autoencoder solves two problems at once. Its objective simultaneously trains a reconstruction and pulls the aggregate distribution of codes towards a chosen prior, so that after training one can sample from the prior and expect the decoder to produce something plausible. That convenience is bought by constraining the representation during training.

The MAE declines the bargain. Equation (19) bounds the magnitude of the codes but says nothing about their shape, so after training the encoder has mapped the corpus somewhere in $\mathbb{R}^{84}$ without any commitment as to where. This leaves a gap exactly where a generative system needs one filled: the encoder records where training molecules landed, but supplies no procedure for drawing a *new* vector likely to decode into a reasonable molecule.

DrugNet closes the gap after the fact. The encoded corpus

$$\mathbf{z}_n = E_\phi(\mathbf{x}_n) \in \mathbb{R}^{84}, \quad \mathbf{Z} = \{\mathbf{z}_n\}_{n=1}^N, \quad (22)$$

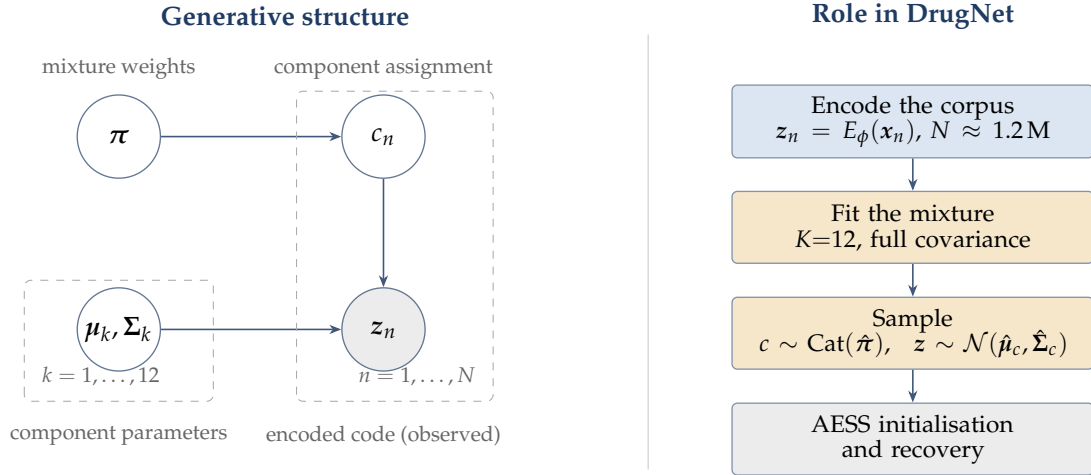


Figure 3: Ex-post latent density estimation. Left: the generative structure of the mixture, in plate notation — mixture weights π and component parameters (μ_k, Σ_k) are global, the assignment c_n is latent, and the encoded code z_n is what the fitting procedure observes. Right: the role this plays in DrugNet, where sampling the fitted mixture supplies the initial AESS carrier population and the replacement vectors used during recovery. The mixture is fitted once, after autoencoder training, and is never updated by the search.

is treated as a sample from an unknown density, and a Bayesian Gaussian mixture [29] is fitted to it. Figure 3 shows the arrangement. The trade is explicit: the representation is left unconstrained during training, at the price of a second estimation stage afterwards, and of a density model that can only ever be as good as its family allows.

6.2 The mixture model

Introduce a discrete component variable $c_n \in \{1, \dots, K\}$ with $K = 12$. The generative story is that a component is drawn from the mixture weights and the code is then drawn from that component’s Gaussian:

$$c_n \mid \pi \sim \text{Cat}(\pi_1, \dots, \pi_K), \quad z_n \mid (c_n = k) \sim \mathcal{N}(\mu_k, \Sigma_k). \quad (23)$$

Marginalising the unobserved assignment gives the density that is actually fitted,

$$p(z_n \mid \pi, \mu, \Sigma) = \sum_{k=1}^K \pi_k \mathcal{N}(z_n \mid \mu_k, \Sigma_k), \quad \sum_{k=1}^K \pi_k = 1, \quad \pi_k \geq 0, \quad (24)$$

where each component is the d -dimensional Gaussian density with $d = 84$,

$$\mathcal{N}(z \mid \mu_k, \Sigma_k) = \frac{1}{(2\pi)^{d/2} |\Sigma_k|^{1/2}} \exp\left[-\frac{1}{2}(z - \mu_k)^\top \Sigma_k^{-1} (z - \mu_k)\right]. \quad (25)$$

Two modelling choices deserve comment. Using $K = 12$ components asserts that the latent cloud is better described as a union of a dozen locally Gaussian regions than as one; this is a much weaker assumption than a single Gaussian prior, and it is what lets the model represent a code distribution with several distinct modes. Using *full* covariance matrices — rather than diagonal or spherical ones — lets each component tilt along correlated directions in the 84-dimensional space. The cost is parameters: a full 84×84 covariance carries $84 \cdot 85/2 = 3,570$ free entries per component, so the

twelve components together account for over 42,000 covariance parameters. With ≈ 1.2 million encoded molecules this is comfortably estimable, but it is the reason a diagonal approximation would have been the obvious economy if the corpus had been smaller.

6.3 Priors, responsibilities, and variational fitting

The distinction from an ordinary maximum-likelihood mixture is that a Bayesian mixture places priors over the parameters and infers a posterior rather than a point estimate. In the conjugate formulation the weights carry a Dirichlet prior and each component carries a Normal–Wishart prior over its mean and precision $\Lambda_k = \Sigma_k^{-1}$:

$$\boldsymbol{\pi} \sim \text{Dir}(\alpha_1, \dots, \alpha_K), \quad \Lambda_k \sim \mathcal{W}(\mathbf{W}_0, \nu_0), \quad \boldsymbol{\mu}_k \mid \Lambda_k \sim \mathcal{N}(\mathbf{m}_0, (\beta_0 \Lambda_k)^{-1}). \quad (26)$$

The practical payoff is regularisation. A maximum-likelihood mixture with full covariances can drive a component onto a single point and send its likelihood to infinity; the Wishart prior penalises the degenerate covariance that requires. The Dirichlet prior does something equally useful for this application: with a small concentration parameter it pulls surplus weights towards zero, so that a mixture fitted with $K = 12$ may end up using fewer *effective* components, and K becomes an upper bound rather than a commitment.

Fitting proceeds by variational inference. The exact posterior over $(\mathbf{C}, \boldsymbol{\pi}, \boldsymbol{\mu}, \Lambda)$ is intractable, so it is approximated by a factorised distribution q chosen to maximise the evidence lower bound,

$$\mathcal{L}_{\text{ELBO}}(q) = \mathbb{E}_q[\log p(Z, \mathbf{C}, \boldsymbol{\pi}, \boldsymbol{\mu}, \Lambda)] - \mathbb{E}_q[\log q(\mathbf{C}, \boldsymbol{\pi}, \boldsymbol{\mu}, \Lambda)] \leq \log p(Z). \quad (27)$$

The gap between the bound and $\log p(Z)$ is the Kullback–Leibler divergence from q to the true posterior, so maximising Equation (27) in q is exactly minimising that divergence.

The optimisation alternates two updates that mirror expectation–maximisation. Given the current parameter distributions, each code is assigned soft responsibilities

$$\log r_{nk} = \text{const} + \mathbb{E}_q[\log \pi_k] + \frac{1}{2} \mathbb{E}_q[\log |\Lambda_k|] - \frac{1}{2} \mathbb{E}_q[(\mathbf{z}_n - \boldsymbol{\mu}_k)^\top \Lambda_k (\mathbf{z}_n - \boldsymbol{\mu}_k)], \quad (28)$$

normalised so that $\sum_k r_{nk} = 1$; this is the Bayesian analogue of the point-estimate responsibility $r_{nk} \propto \pi_k \mathcal{N}(\mathbf{z}_n \mid \boldsymbol{\mu}_k, \Sigma_k)$. Given responsibilities, the parameter distributions are updated from the weighted sufficient statistics

$$N_k = \sum_{n=1}^N r_{nk}, \quad \bar{\mathbf{z}}_k = \frac{1}{N_k} \sum_{n=1}^N r_{nk} \mathbf{z}_n, \quad \mathbf{S}_k = \frac{1}{N_k} \sum_{n=1}^N r_{nk} (\mathbf{z}_n - \bar{\mathbf{z}}_k)(\mathbf{z}_n - \bar{\mathbf{z}}_k)^\top. \quad (29)$$

Appendix B gives the update equations in full.

6.4 Sampling

Once fitted, the mixture is used only as a sampler. Drawing a latent vector is two steps,

$$c \sim \text{Cat}(\hat{\boldsymbol{\pi}}), \quad \mathbf{z} \sim \mathcal{N}(\hat{\boldsymbol{\mu}}_c, \hat{\Sigma}_c), \quad (30)$$

where the hatted quantities are the fitted posterior means. AESS calls this in two places: once to initialise all 400 carriers, and again whenever recovery replaces a carrier’s solution. Starting a population from Equation (30) rather than from an arbitrary distribution is what makes the initial candidates decodable — a vector drawn from, say, a broad uniform distribution over $\mathbb{R}^{84}$ would fall far from anything the decoder saw during training.

6.5 Configuration and what the implementation fixes

The implementation constructs `BayesianGaussianMixture` with `n_components=12` and `covariance_type='full'`, with `warm_start=True`, and then loads a previously fitted model from disk. Because no `fit` call appears in the supplied notebook, the convergence history of the fitted object cannot be recovered from the file.

Two consequences follow, and both are stated rather than assumed away. First, every prior hyperparameter in Equation (26) is left at its library default, so the numbers $\alpha_0, \mathbf{m}_0, \beta_0, \mathbf{W}_0, \nu_0$ are not project choices and cannot be quoted as such. Second — and this is a genuine ambiguity in the reconstruction — the library’s default weight prior is a Dirichlet *process* (stick-breaking) construction rather than the finite Dirichlet distribution of Equation (26). Since the construction argument is not set explicitly, the fitted sampler most likely used the stick-breaking variant. The finite-Dirichlet form is presented above because it is the standard pedagogical case and because the report describes a fixed twelve-component mixture; the stick-breaking default is recorded here so that a re-implementation does not silently reproduce something different. The report’s statement of 100 training iterations coincides with the library’s default iteration cap, which is suggestive but not evidence.

Table 6: BGMM configuration. Prior hyperparameters are library defaults rather than project choices and are therefore not tabulated as project settings.

Item	Value	Source
Mixture components K	12	code
Covariance structure	full	code
Warm start	enabled	code
Fitted model	loaded from a serialised file	code
Weight prior	library default (stick-breaking; not set explicitly)	inferred
Training iterations	100	report

7 Advanced Evolutionary Spread Search

7.1 The algorithm as a population process

AESS is a stochastic population process over latent vectors. A population of $N_c = 400$ *carriers* is maintained; carrier i at iteration t holds a latent solution $\mathbf{z}_i^{(t)} \in \mathbb{R}^{84}$, a class $q_i \in \{H, M, S, C\}$ (healthy, mild, severe, critical), a stored intensity, and a scalar fitness $f(\mathbf{z}_i)$ that the algorithm minimises. For the target-affinity objective the fitness is the negated DeepADTP prediction, so a larger predicted affinity corresponds to a smaller fitness.

The four classes are not a model of anything biological. They are search-policy states, and the only thing that distinguishes them is the proposal distribution applied to a carrier’s latent vector. The critical carrier is an elitist memory of the best solution found so far and is never perturbed. Severe carriers make multiplicative local perturbations. Mild carriers are pulled towards the critical solution while retaining stochastic movement. Healthy carriers are pushed outwards, away from the region already covered.

One iteration applies six operators in a fixed order: mutation, infection, recovery, adaptation, and selection, with assignment performed once before the loop begins. Figure 4 shows the cycle and the class-specific update rules; Algorithm 1 states the whole procedure.

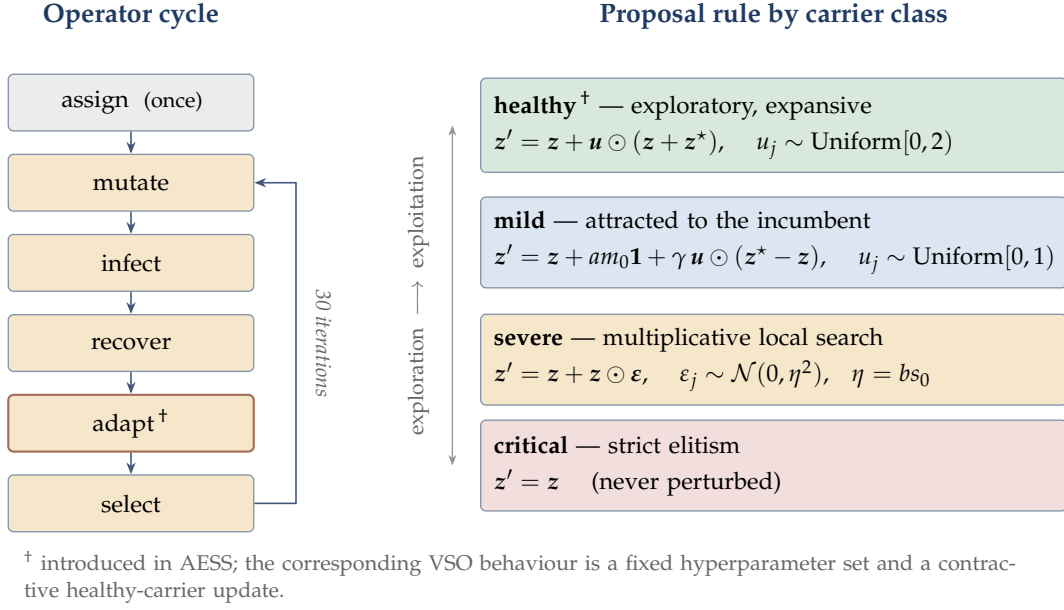


Figure 4: AES operator cycle and the class-dependent proposal rules. Left: the fixed order in which operators are applied within one iteration, with the two novel mechanisms marked. Right: what each carrier class does to its latent vector, arranged from most exploratory (top) to fully elitist (bottom). The healthy rule is expansive rather than contractive, which is the design choice examined in Section 7.7 and again in Section 13.

7.2 Assignment and selection

Assignment runs once. Every carrier receives a latent vector drawn from the fitted mixture,

$$z_i^{(0)} \sim p_\psi(z), \quad i = 1, \dots, 400, \quad (31)$$

and is placed in the healthy class. The intent is broad initial coverage of the region the encoder actually populated, before selection pressure begins to concentrate the population.

Selection evaluates every carrier under the current fitness function and promotes the minimiser to critical:

$$z^* \leftarrow z_{i^*}, \quad i^* = \arg \min_i f(z_i). \quad (32)$$

When a new critical carrier is found the previous one is demoted to severe, the critical-carrier age is reset to zero, and the adaptation state is restored to its initial values. When no improvement occurs, the age is incremented. That counter is the only feedback signal in the algorithm, and Section 7.6 is about what is done with it.

7.3 Mutation

Mutation is where the class distinction becomes concrete. All random vectors are drawn per-coordinate in $\mathbb{R}^{84}$; the implementation is explicit about this, which resolves a scalar-versus-vector ambiguity in the report’s notation.

Healthy carriers. With u a vector of independent uniform draws,

$$z' = z + u \odot (z + z^*), \quad u_j \sim \text{Uniform}[0, 2). \quad (33)$$

Mild carriers. The step combines a fixed base intensity with attraction towards the critical solution,

$$\Delta = a m_0 \mathbf{1} + \gamma \mathbf{u} \odot (\mathbf{z}^* - \mathbf{z}), \quad u_j \sim \text{Uniform}[0, 1), \quad \mathbf{z}' = \mathbf{z} + \Delta, \quad (34)$$

with $a = 1$ and $\gamma = 2$. The second term is a directed exploitation step whose magnitude shrinks as a carrier approaches the incumbent, so mild carriers refine without collapsing onto it exactly.

Severe carriers. The step is multiplicative and Gaussian,

$$\eta = b s_0, \quad \varepsilon_j \sim \mathcal{N}(0, \eta^2), \quad \mathbf{z}' = \mathbf{z} + \mathbf{z} \odot \varepsilon, \quad (35)$$

with decay coefficient $b = 0.9$. Because the perturbation scales with the carrier's own coordinates, a severe carrier explores proportionally: coordinates that are already large move further in absolute terms than coordinates near zero.

Critical carrier. No update. Elitism is enforced by the absence of a branch rather than by an explicit rule.

The base intensities are set once per carrier at construction. The severe base is $s_0 = 1/U$ with $U \sim \text{Uniform}(0, 1)$, a reciprocal-uniform variable supported on $[1, \infty)$ whose mean does not exist; the severe standard deviation $\eta = 0.9 s_0$ is therefore at least 0.9 and occasionally very much larger. That heavy tail is not incidental — it is what gives severe carriers an occasional long jump, and it is worth knowing about when reasoning about run-to-run variance.

The mild base is $m_0 = (\text{first dimension of the solution array})/10$. Since mixture samples are returned with shape $(1, 84)$, this evaluates to 0.1 for every carrier rather than the $84/10 = 8.4$ that the expression appears to intend. [Appendix C](#) records this; the effect is that the constant component of the mild step is an order of magnitude smaller than intended, so mild carriers rely almost entirely on their attraction term.

7.4 Infection

Infection transfers information from stronger carriers into weaker ones. Infected carriers are sorted best-first under the minimisation fitness; healthy carriers are sorted worst-first. Each infected carrier then contacts up to H of the currently worst healthy carriers — so the information flows into the part of the population that has least to lose.

For an infector of class q , infection of a contacted carrier is a Bernoulli event with class-dependent probability

$$\rho_M = 0.20, \quad \rho_S = 0.50, \quad \rho_C = 0.80. \quad (36)$$

Conditional on infection, the target becomes mild or severe with the class-dependent probabilities in [Table 7](#). The two outcomes are qualitatively different operators. A newly mild carrier receives a Bernoulli crossover: each of the 84 coordinates is independently copied from the infector with probability $\frac{1}{2}$,

$$z'_j = \begin{cases} z_j^{\text{inf}} & \text{with probability } \frac{1}{2}, \\ z_j & \text{otherwise,} \end{cases} \quad j = 1, \dots, 84, \quad (37)$$

which is uniform crossover in the genetic-algorithm sense and produces a genuine recombination of two solutions. A newly severe carrier instead receives the infector’s vector wholesale, $\mathbf{z}' = \mathbf{z}^{\text{inf}}$, which is not recombination but replication — a much stronger exploitation step that reduces population diversity by one.

Table 7: Class transition probabilities on infection. A stronger infector is more likely to produce a severe target, and severe targets copy the infecting vector in full, so information from better carriers propagates more aggressively.

Infecting class	Target becomes mild	Target becomes severe
Mild	0.70	0.30
Severe	0.50	0.50
Critical	0.30	0.70

Probabilistic infection matters for a reason that is easy to miss: it stops the population collapsing deterministically onto one point in a single pass. With $\rho_C = 0.8$ and $H = 20$, the critical carrier alone can convert sixteen healthy carriers per iteration, and the severe branch replicates its vector exactly. Without the Bernoulli gate and the mild crossover branch, one iteration would be enough to homogenise the population.

7.5 Recovery

Recovery is the counterweight. It activates only when every carrier in the population has been infected, which is to say when no exploratory carriers remain at all. It then takes the worst $\lfloor r_{\text{rec}} N_C \rfloor$ infected carriers, replaces each of their solutions with a fresh draw from the fitted mixture, and moves each one class back towards health: severe becomes mild, mild becomes healthy.

The trigger condition is strict, and it is worth being precise about the consequence. Recovery is not a steady diversity-maintenance mechanism operating every iteration; it is an all-or-nothing rescue that fires only after the infection operator has consumed the entire healthy population. In runs where a few healthy carriers persist, recovery never runs at all, and the only source of new diversity is the stochasticity in the mutation rules.

7.6 Adaptation

The first of the two modifications that distinguish AECS from VSO is a controller that changes the search hyperparameters while the search is running. The control signal is the critical-carrier age a : the number of consecutive selection cycles for which the identity of the best carrier has not changed. A large a is read as evidence that local exploitation has stopped paying, and the response is to increase exploration.

The coefficient is a logistic function of age centred at nine,

$$p(a) = \frac{1}{1 + \exp(9 - a)} = \sigma(a - 9) \in (0, 1). \quad (38)$$

The choice of a logistic rather than a linear ramp gives the controller a deliberate dead zone. For $a \leq 4$ the coefficient is below 0.007 and adaptation is effectively off, so short-lived plateaus — which are normal in any stochastic search — provoke no reaction. The response then rises steeply through $p(9) = 0.5$ and saturates near one by $a \approx 14$. In effect the controller waits about nine iterations before concluding that a plateau is real, then

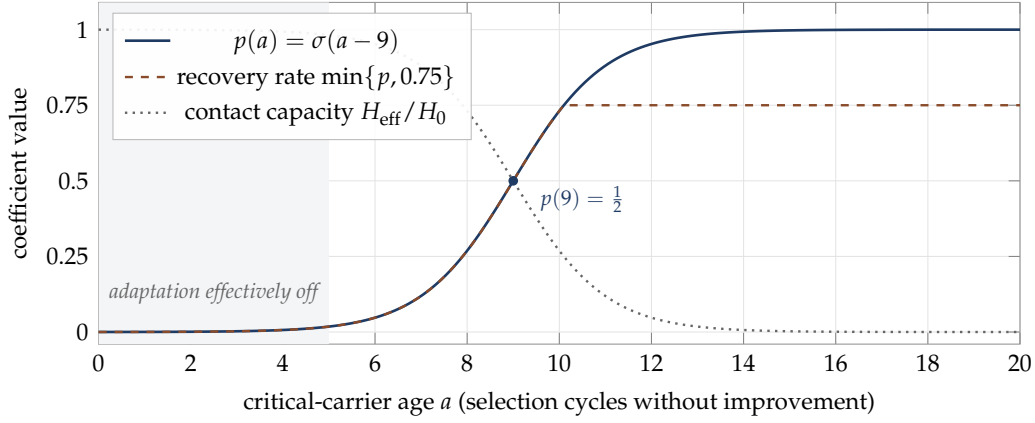


Figure 5: The adaptation controller. The logistic coefficient $p(a) = \sigma(a - 9)$ governs four hyperparameters at once; the dead zone below $a \approx 5$ means ordinary short plateaus provoke no response, while sustained stagnation switches the search into an exploratory regime over roughly five iterations. Contact capacity is shown normalised by $H_0 = 20$. Since a run lasts thirty iterations, the transition region occupies the middle of a run rather than its tail.

switches regime over roughly five more. Given that a run is only thirty iterations long, this places the transition squarely in the middle of a run.

Four hyperparameters are driven by $p(a)$:

$$r_{\text{rec}} = \min\{p(a), 0.75\}, \quad (39)$$

$$\gamma_{\text{eff}} = 1 + p(a), \quad (40)$$

$$H_{\text{eff}} = \lfloor H_0 (1 - p(a)) \rfloor, \quad (41)$$

$$\rho_q \leftarrow \frac{\rho_q}{1 + p(a)}, \quad q \in \{\text{M}, \text{S}, \text{C}\}. \quad (42)$$

These pull in a consistent direction. Shrinking the contact cap H and dividing down the infection probabilities both slow the conversion of healthy carriers, so more of the population stays exploratory. Raising the recovery rate returns more carriers to health when recovery does fire. Raising γ enlarges the mild carriers' stochastic step. Figure 5 plots the curve and the three derived quantities together.

The report also describes adapting the severe decay coefficient b upwards with p . The implementation computes this quantity as $\min(\text{int}(\text{init_decay} + p/10), 0.99)$, and the integer cast makes the expression evaluate to zero for every $p \in [0, 1)$ given $b_0 = 0.9$ — which would set the severe mutation scale to zero and freeze severe carriers entirely. The described intent and the written code disagree here, so the description above follows the report and Appendix C records the code exactly.

7.7 The redesigned healthy-carrier update

The second modification concerns what healthy carriers do. In VSO, a healthy carrier that is not infected is effectively re-randomised. The report identifies two weaknesses in that behaviour: re-randomising from the same distribution repeatedly re-samples regions that have already been searched, and it can never move the population into latent regions that lie outside the empirical density but that the decoder may still handle.

The replacement rule is Equation (33), and its character is worth making explicit

Algorithm 1 Advanced Evolutionary Spread Search (one run)

Require: fitted mixture p_ψ , fitness f , population size N_c , iterations T , $H_0, r_0, \{\rho_q\}, a, \gamma_0, b_0$

- 1: **assign:** $z_i \sim p_\psi(z), q_i \leftarrow H$ for $i = 1, \dots, N_c$
- 2: **select:** evaluate $f(z_i)$ for all i ; set $z^* \leftarrow z_{\arg \min_i f}$, its class to C , age $\leftarrow 0$
- 3: **for** $t = 1$ **to** T **do**
- 4: **mutate:** apply Equations (33) to (35) by class; the critical carrier is unchanged
- 5: **infect:** sort infected best-first and healthy worst-first; each infected carrier contacts $\min(H_{\text{eff}}, |\mathcal{H}|)$ healthy carriers, infecting each with probability ρ_q and converting it to mild (coordinate crossover, Equation (37)) or severe (full copy)
- 6: **recover:** **if** no healthy carriers remain **then** resample the worst $\lfloor r_{\text{rec}} N_c \rfloor$ infected carriers from p_ψ and demote each one class towards health
- 7: **adapt:** $p \leftarrow \sigma(\text{age} - 9)$; update $r_{\text{rec}}, \gamma_{\text{eff}}, H_{\text{eff}}, \{\rho_q\}$ by Equations (39) to (42)
- 8: **select:** age $\leftarrow \text{age} + 1$; re-evaluate all carriers; **if** a better solution is found **then** promote it to critical, demote the old critical to severe, set age $\leftarrow 0$ and restore the adaptation state
- 9: **end for**
- 10: **return** the severe and critical carriers

rather than leaving in the notation. Taking expectations coordinate-wise, with $\mathbb{E}[u_j] = 1$,

$$\mathbb{E}[z' \mid z, z^*] = z + \mathbb{E}[u] \odot (z + z^*) = 2z + z^*. \quad (43)$$

The update is *expansive*. In expectation a healthy carrier’s coordinates double each time it is mutated, plus an offset towards the critical vector, so over T iterations the magnitude grows on the order of 2^T . Contrast the corresponding VSO rule, which is built on a *difference* ($z^* - z$) and therefore contracts towards the incumbent; changing that sign is precisely what converts a convergent operator into a divergent one.

A reference re-implementation of the operators, supplied with this document and driven by a synthetic objective so that no trained component is involved, makes the size of the effect concrete: over thirty iterations with 400 carriers, the population’s mean latent norm grew by factors ranging from about 4 to more than 10^{24} depending only on the random seed. That spread is itself the finding. How far the search travels is decided by whether a given healthy carrier happens to escape infection for a long run of iterations, not by anything about the objective being optimised.

This is what the report means by moving away from already-explored regions, and as a mechanism for escaping the empirical density it works: within a handful of iterations healthy carriers are far outside the support of p_ψ . Two things follow, and both are load-bearing for how the results in Section 11 should be read. Healthy carriers rapidly reach latent vectors on which the decoder is extrapolating well outside its training distribution, where Equation (16) offers no guarantee that the decoded token sequence corresponds to anything chemically sensible. And because the growth is geometric rather than tuned, the distance travelled is governed by iteration count rather than by any property of the objective. Section 13 takes up what this implies for the Penalized LogP result specifically.

7.8 Hyperparameters

Table 8: AESS configuration. All values are read from the implementation’s constructor call and from the search loop; the run and iteration counts come from the report.

Parameter	Value	Source
Population size N_c	400	code
Iterations per run T	30	report
Initial recovery rate r_0	0.30	code
Contact cap H_0	20	code
Mild base coefficient a	1	code
Mild attraction γ	2	code
Severe decay b	0.9	code
Infection probabilities $\rho_{M/S/C}$	0.20 / 0.50 / 0.80	code
Transition probabilities	Table 7	code
Adaptation coefficient	$p(a) = \sigma(a - 9)$	code
Recovery cap under adaptation	0.75	code
Healthy random vector	84-D, Uniform[0, 2)	code
Mild random vector	84-D, Uniform[0, 1)	code
Severe base intensity	$s_0 = 1/U, U \sim \text{Uniform}(0, 1)$	code
Mild base intensity	$m_0 = 0.1$ (see Appendix C)	code

8 DeepADTP: attention-based drug–target prediction

8.1 Objective

DeepADTP takes two sequences — a protein’s amino-acid sequence and a molecule’s SELFIES representation — and returns a scalar estimate of interaction strength. Inside the integrated system that scalar is the only thing telling AESS whether one generated molecule is preferable to another. [Figure 6](#) shows the architecture.

The design has two symmetric feature extractors and one interaction module. Each sequence is embedded, passed through a stack of dilated causal convolutions with residual connections, and then related to the other sequence by attention in both directions before regression.

8.2 Dilated convolution and the receptive field

A one-dimensional convolution with dilation d combines positions separated by d rather than adjacent ones. For kernel width K , input sequence x and activation φ ,

$$y_t = \varphi \left(\mathbf{b} + \sum_{j=0}^{K-1} \mathbf{w}_j x_{t-dj} \right). \quad (44)$$

The implementation uses causal padding, so index $t - dj$ never reaches beyond position t and positions past the end of the real sequence are zero.

The reason for dilation is arithmetic. A stack of L ordinary convolutions of width K has receptive field $1 + L(K - 1)$, which grows linearly in depth; with $K = 6$, reaching a few hundred protein positions would require dozens of layers. Doubling the dilation at each level instead gives

$$R_L = 1 + (K - 1) \sum_{\ell=0}^{L-1} d_\ell = 1 + (K - 1) (2^L - 1) \quad (45)$$

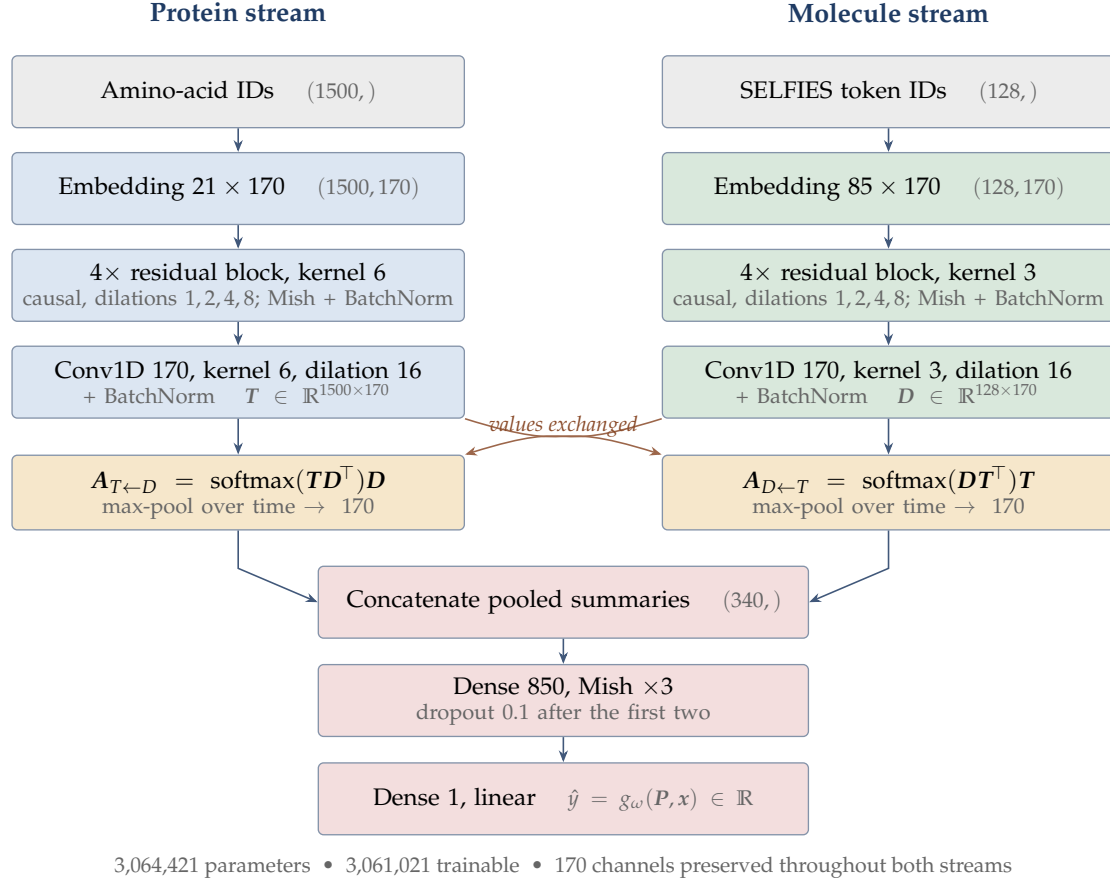


Figure 6: DeepADTP. The protein and molecule streams are structurally identical apart from kernel width (6 against 3, reflecting the different granularity of amino-acid and SELFIES tokens) and sequence length. Both preserve 170 channels throughout, which is what makes the bidirectional dot-product attention between them well defined without a projection. The dilation schedule 1,2,4,8,16 expands the receptive field geometrically, reaching 156 protein positions by the final layer.

for the schedule $d_\ell = 2^\ell$. For the protein branch, $K = 6$ and five layers with $d \in \{1,2,4,8,16\}$ give $R_5 = 1 + 5 \times 31 = 156$ positions; for the molecule branch, $K = 3$ gives $R_5 = 63$ positions out of a 128-token sequence. Five layers therefore cover a substantial fraction of each sequence with roughly $170 \times 170 \times K$ parameters per layer, which is the trade dilation buys [18].

The activation throughout both stacks is Mish [30],

$$\text{mish}(x) = x \tanh(\text{softplus}(x)) = x \tanh(\log(1 + e^x)), \quad (46)$$

a smooth non-monotonic function that, unlike ReLU, has a small negative lobe and a continuous derivative everywhere.

8.3 Residual blocks

Each of the first four layers in a branch is wrapped in a residual block [19]. Writing $\mathcal{F}$ for the dilated convolution followed by batch normalisation [31], the block computes

$$h^{(\ell+1)} = h^{(\ell)} + \mathcal{F}(h^{(\ell)}). \quad (47)$$

The block therefore learns a correction to the representation it received rather than a replacement, and gradients reach earlier layers through the identity path without passing through $\mathcal{F}$. This is what makes the addition in Equation (47) well typed at all: every layer in both branches keeps exactly 170 channels, so input and output shapes match and no projection shortcut is needed. The fifth layer applies the dilation-16 convolution and batch normalisation without a residual add.

8.4 Bidirectional attention

Let $D \in \mathbb{R}^{128 \times 170}$ be the processed molecular feature sequence and $T \in \mathbb{R}^{1500 \times 170}$ the processed protein sequence. The implementation applies unscaled dot-product attention [20] in both directions, taking the key equal to the value:

$$A_{D \leftarrow T} = \text{softmax}(DT^\top) T \in \mathbb{R}^{128 \times 170}, \quad A_{T \leftarrow D} = \text{softmax}(TD^\top) D \in \mathbb{R}^{1500 \times 170}, \quad (48)$$

where the softmax is taken row-wise. The two directions ask complementary questions. $A_{D \leftarrow T}$ asks, for each molecular position, which protein features are most relevant to it — a molecule-centred view of the interaction. $A_{T \leftarrow D}$ asks the reverse. Computing both means the interaction representation is not forced to privilege one sequence as the query.

The output of Equation (48) is a weighted average of value vectors, not a rescaling of the query; this is worth stating because the schematic attention expression in the original report is ambiguous on the point. Each attended sequence is then reduced by global max-pooling to 170 features, and the two pooled vectors are concatenated into a 340-dimensional joint representation.

8.5 Regression head and objective

The joint representation passes through three dense layers of $5 \times 170 = 850$ units with Mish activations, with dropout [32] at rate 0.1 after the first and second, and then a single linear unit:

$$\hat{y} = g_\omega(\mathbf{P}, \mathbf{x}) = \mathbf{w}^\top \mathbf{h}^{(3)} + b \in \mathbb{R}. \quad (49)$$

Training minimises mean squared error over the KIBA pairs,

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2. \quad (50)$$

The model holds 3,064,421 parameters, of which 3,061,021 are trainable and 3,400 are batch-normalisation statistics.

Remark 8.1. Both embeddings are constructed with masking enabled, but convolutional layers do not consume Keras masks. Padded positions therefore participate in the convolutions and in the batch-normalisation statistics. For the 3CL protease this is not a marginal effect: 1194 of 1500 protein positions are padding, so the great majority of the protein stream’s positions carry no residue. The model was trained under the same convention, so the padding is at least consistent between training and inference; but a re-implementation should apply explicit masking rather than assume the behaviour is benign.

Table 9: DeepADTP configuration. As with the MAE, the supplied notebook constructs the model and loads trained weights but does not contain the training call, so the optimisation settings are reported rather than code-verified.

Item	Value	Source
Molecule input length	128 SELFIES tokens	code
Protein input length	1500 amino-acid positions	code
Molecule embedding	85×170	code
Protein embedding	21×170	code
Feature width	170 channels throughout	code
Molecule kernel width	3	code
Protein kernel width	6	code
Dilation schedule	1, 2, 4, 8 residual; 16 final	code
Padding	causal	code
Activation	Mish	code
Normalisation	batch normalisation in every block	code
Interaction	bidirectional unscaled dot-product attention	code
Pooling	global max over time, per direction	code
Dense head	850×3 , Mish; dropout 0.1 after the first two	code
Output	single linear unit	code
Loss	mean squared error	report
Optimiser / learning rate	Adam, 5×10^{-4}	report
Epochs / batch size	100 / 180	report
Total parameters	3,064,421	code

9 The integrated pipeline

9.1 Training, then search

The two neural models are trained separately with Adam [28]. Once the MAE has converged its encoder is applied to the whole molecular corpus, and the mixture of Section 6 is fitted to the resulting codes. At that point all four capabilities the system needs are in place: the MAE defines the map between chemical and latent space, the mixture defines where in latent space to start, DeepADTP defines the objective, and AESS defines how to move.

Nothing in the search phase updates any learned parameter. The autoencoder, the mixture and the affinity model are all frozen, which is what makes a single trained system reusable across different targets and different objectives — and also what fixes, once and for all, the region of chemical space the decoder can express.

9.2 One optimisation iteration

Concretely, one iteration of the target-affinity search does the following for each of the 400 carriers. The carrier’s latent vector is decoded to a $(128, 85)$ tensor of token probabilities and reduced to a token sequence by Equation (17). That sequence is paired with the 306-residue 3CL protease sequence, zero-padded to 1500, and passed to DeepADTP. The returned scalar is negated to give the minimisation fitness of Equation (5). AESS then applies its operators and the loop repeats.

The same machinery supports two other objectives by substituting the scoring function. For drug-likeness,

$$f_{\text{QED}}(z) = -\text{QED}(\text{Mol}(D_{\theta}(z))), \quad (51)$$

and for the penalised octanol–water partition coefficient,

$$f_{\text{PlogP}}(\mathbf{z}) = -J_{\text{PlogP}}(\text{Mol}(D_{\theta}(\mathbf{z}))), \quad (52)$$

where $\text{Mol}(\cdot)$ denotes decoding the SELFIES string and parsing it with RDKit [33]. When parsing fails, the implementation returns 0 for QED and -10 for Penalized LogP, so an unparseable candidate is scored as poor rather than discarded. This detail matters more than it looks: it means the search never sees a hard constraint, only a penalty, and a region of latent space that decodes to nothing valid is merely unattractive rather than unreachable.

9.3 Why the surrogate structure matters

Running a physical measurement inside every search iteration is impossible; running a docking simulation for $400 \times 30 \times 15 = 180,000$ candidate evaluations would already be expensive. Replacing the objective with a learned model that evaluates in milliseconds is what makes population-based search feasible at all, and this substitution — expensive objective, cheap learned proxy, search the proxy — is a standard pattern in optimisation.

Its cost is equally standard, and stating it precisely is necessary for reading [Section 11](#). AESS optimises whatever DeepADTP rewards. Where the affinity model is accurate, the search is pursuing genuine structure in the objective; where it is wrong, the search will exploit the error just as efficiently, and a population-based optimiser is unusually good at finding exactly those regions. This does not diminish the reported numbers, but it fixes their meaning: the 308 retained molecules are candidates under a surrogate, and the surrogate’s reliability off its training distribution is not something the project measured.

10 Experimental protocol

10.1 Generative evaluation of the autoencoder

The MAE was evaluated with the MOSES molecular-generation benchmark [34] and compared against published figures for JT-VAE [6], LatentGAN [7], a VAE [16] and an adversarial autoencoder [35]. Four metrics are used.

Validity is the fraction of generated token sequences that parse to a chemically valid molecule. *Uniqueness@10k* is the fraction of distinct molecules within a sample of 10,000, which detects a generator that produces valid output by repeating itself. *Novelty* is the fraction of generated molecules absent from the training set, which detects one that has memorised its corpus. *Internal diversity* measures structural spread within the generated set as one minus the mean pairwise Tanimoto similarity,

$$\text{IntDiv} = 1 - \frac{2}{|G|(|G| - 1)} \sum_{i < j} T(\mathbf{fp}_i, \mathbf{fp}_j), \quad T(\mathbf{u}, \mathbf{v}) = \frac{|\mathbf{u} \cap \mathbf{v}|}{|\mathbf{u} \cup \mathbf{v}|}, \quad (53)$$

where $\mathbf{fp}_i$ is a binary molecular fingerprint and T is the Tanimoto coefficient: the number of bits set in both fingerprints divided by the number set in either. The four metrics are informative jointly rather than individually, since each is trivially maximised at the expense of the others — a generator that emits one memorised molecule scores validity 1 and everything else 0.

One methodological qualification applies and should be stated in full. The project did not retrain the MAE on the MOSES training corpus; it evaluated a model trained on its own ≈ 1.2 -million-molecule corpus, against a MOSES setup of roughly 1.6 million

molecules, and computed novelty relative to its own training set. The comparison is therefore a benchmark-level comparison under the established MOSES metrics rather than a controlled same-training-data ablation. That distinction constrains what can be attributed to the architecture specifically; it does not make the evaluation informal, and the metrics themselves are the standard ones.

10.2 Affinity evaluation

DeepADTP was compared against SimBoost [36], DeepDTA [12] and AttentionDTA [13] on KIBA, using mean squared error and the concordance index [37].

The two metrics measure different things, and reporting both is not redundant. MSE, given in Equation (50), measures calibration on the label scale: it asks whether the predicted number is close to the measured number, and squaring makes it dominated by the largest errors. The concordance index measures ranking:

$$\text{CI} = \frac{1}{|\mathcal{P}|} \sum_{(i,j) \in \mathcal{P}} [\mathbb{1}(\hat{y}_i > \hat{y}_j) + \frac{1}{2}\mathbb{1}(\hat{y}_i = \hat{y}_j)], \quad \mathcal{P} = \{(i,j) : y_i > y_j\}, \quad (54)$$

the fraction of genuinely ordered pairs that the model orders correctly, with ties counted as half. A CI of 0.5 is chance and 1.0 is a perfect ordering.

The distinction matters for DrugNet specifically. A model could be badly calibrated — systematic offset, wrong scale — and still rank candidates perfectly, and ranking is all AESS needs, since Equation (5) is used only for comparison between carriers. Conversely a model with low MSE achieved by predicting close to the mean everywhere would be useless as a search objective. The two numbers together say whether the model is both accurate and discriminative.

10.3 Integrated target search

The full system was run fifteen times against the 3CL protease objective, thirty iterations per run. At the end of each run the severe and critical carriers were retained — that is, the exploitative part of the population rather than the exploratory part — yielding a pool of 308 molecules across all runs. Their predicted fitness distribution was compared with that of the COVID-19 molecule set drawn from ChEMBL.

10.4 Property optimisation and the VSO ablation

The optimisation machinery was also tested on two standard molecular-property objectives, run ten times for thirty iterations each. The QED objective uses the RDKit implementation [24, 33]. The Penalized LogP objective is the one introduced with GCPN [10], and the implementation reproduces it exactly:

$$J_{\text{PlogP}}(\text{mol}) = \frac{\log P - \mu_{\log P}}{\sigma_{\log P}} + \frac{\text{SA} - \mu_{\text{SA}}}{\sigma_{\text{SA}}} + \frac{\text{cycle} - \mu_{\text{cycle}}}{\sigma_{\text{cycle}}}, \quad (55)$$

where $\log P$ is the octanol–water partition coefficient, SA is the *negated* synthetic-accessibility score (so that easier-to-synthesise molecules score higher), and the ring term is

$$\text{cycle} = -\max\{0, L_{\max} - 6\}, \quad (56)$$

with $L_{\max}$ the size of the largest ring in the molecular graph’s cycle basis. Each term is standardised by constants computed on ZINC-250k and hard-coded in the implementation: $\mu_{\log P} = 2.4570953$, $\sigma_{\log P} = 1.4343244$; $\mu_{\text{SA}} = -3.0525811$, $\sigma_{\text{SA}} = 0.8335207$; $\mu_{\text{cycle}} = -0.0485697$, $\sigma_{\text{cycle}} = 0.2860212$.

To isolate the contribution of the AESS modifications, the identical DrugNet framework was run a second time with unmodified VSO under the same hyperparameters. This ablation is the only experiment in the project that separates the search algorithm from the rest of the architecture, which makes it the most informative of the reported comparisons.

11 Results

11.1 Molecular generation

Table 10: Reported molecular-generation metrics under the MOSES framework. The MAE was trained on the DrugNet corpus rather than the MOSES corpus, so this is a benchmark-level comparison rather than a same-data ablation; novelty in particular is computed against a different training set for the MAE row than for the published baselines. Best value in each column in bold.

Model	Validity	Uniqueness@10k	Novelty	Internal diversity
MAE (this work)	1.0000	0.9957	0.9560	0.8898
JT-VAE [6]	1.0000	0.9996	0.9143	0.8551
LatentGAN [7]	0.8966	0.9968	0.9498	0.8565
VAE [16]	0.9767	0.9984	0.6949	0.8558
AAE [35]	0.9368	0.9973	0.7931	0.8557

The MAE reports perfect validity, matching JT-VAE and exceeding the three SMILES-based models. That result is largely representational: SELFIES makes essentially any token sequence decode to a valid molecule, so validity near one is what the representation was chosen to deliver rather than evidence about the network. The more informative comparison is what accompanies it. Uniqueness@10k of 0.9957 is marginally below the best figure in the table, but novelty of 0.9560 and internal diversity of 0.8898 are the highest reported values in both columns.

The combination is the point. Perfect validity is easy for a generator that has collapsed onto a narrow region of chemical space, and such a generator would show it in low novelty — the VAE row, with novelty 0.6949, is what that looks like. The MAE achieves validity without that collapse, which supports the claim that the decoder is producing a broad set of molecules rather than recycling its corpus.

11.2 Drug–target affinity

Table 11: Reported KIBA results. Lower MSE indicates better calibration on the label scale; higher CI indicates better ranking of interaction strengths. Both are as given in the project’s comparison table.

Model	MSE ↓	CI ↑
SimBoost [36]	0.222	0.836
DeepDTA [12]	0.194	0.863
AttentionDTA [13]	0.155	0.882
DeepADTP (this work)	0.121	0.890

DeepADTP reports the lowest MSE and the highest CI in the comparison. Against AttentionDTA, the strongest neural baseline listed, MSE is lower by 0.034 and CI higher

by 0.008. Those two differences are of very different relative size — roughly 22% on MSE against roughly 0.9% on CI — and the asymmetry is informative rather than a discrepancy. CI is a bounded rank statistic already close to its ceiling for every listed model, so it compresses; MSE is unbounded below and keeps resolving improvements in calibration after the ranking has largely saturated. The reasonable reading is that DeepADTP’s advantage lies mostly in how well-calibrated its predictions are, with a smaller improvement in ordering.

For DrugNet’s purposes the CI column is arguably the more relevant of the two, since AESS uses the prediction only comparatively. On that metric the improvement over the attention baseline is real but modest.

11.3 Integrated target search

Table 12: Predicted target fitness for the 308 molecules retained from the severe and critical carriers across fifteen runs, against the comparison COVID-19 molecule set from ChEMBL. Both columns are DeepADTP predictions on the same 3CL protease target; neither is a measurement.

Molecule set	Mean predicted fitness	Best predicted fitness
DrugNet retained pool ($n = 308$)	14.9547	16.6439
COVID-19 ChEMBL set	11.2692	13.5748
Difference	+3.6855	+3.0691

The retained pool scores 3.6855 higher on mean predicted fitness than the comparison set, about 33% relative to the comparison mean, and the best single candidate scores 3.0691 higher, about 23%. Both percentages are arithmetic on the reported values rather than separate measurements.

The claim these numbers support is precise and narrower than the original report’s phrasing. DrugNet generated 308 candidate molecules that its own learned fitness

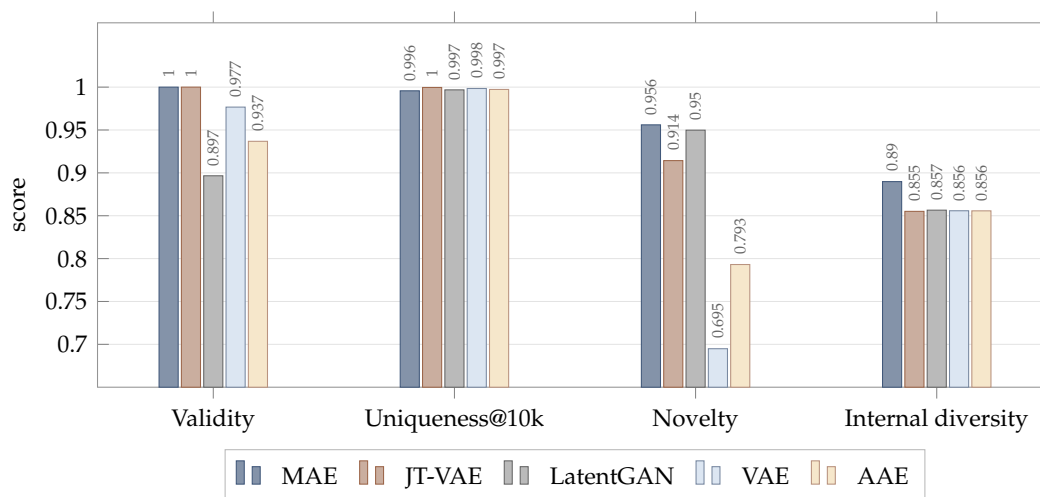


Figure 7: The MOSES metrics from Table 10, plotted on a common axis truncated at 0.65 to make the differences legible. Validity and uniqueness are near-saturated for every model, so the comparison is decided by novelty and internal diversity, where the spread between models is much larger.

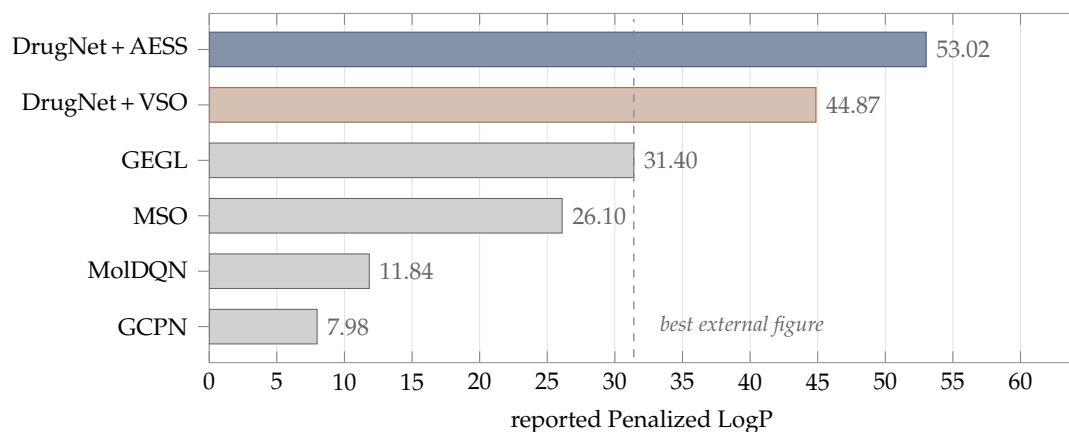


Figure 8: Reported Penalized LogP scores. The two DrugNet bars differ only in the search algorithm. The scale of the gap over the external methods is itself the reason [Section 13](#) examines what the objective rewards, rather than reading the ordering as a straightforward statement about molecular quality.

function scores higher, for the selected target, than the molecules in the comparison pool. Whether that ordering survives contact with biochemical inhibition, cellular antiviral activity, selectivity, pharmacokinetics or toxicity is outside the experiment entirely — and the comparison is against a set of molecules that were themselves selected by a different process, not against a random baseline, which is what makes it interesting rather than trivial.

11.4 Property optimisation and the search ablation

Table 13: Reported property-optimisation results. The final two columns are the same DrugNet framework differing only in the search algorithm, which makes their comparison the project’s cleanest ablation.

Objective	GCPN	MolDQN	MSO	GEGL	DrugNet + VSO	DrugNet + AESS
Penalized LogP	7.98	11.84	26.1	31.40	44.87	53.02
QED	0.948	0.948	0.948	0.948	0.948	0.948

On Penalized LogP, DrugNet with AESS reports 53.02 against 31.40 for the strongest external figure in the table, a difference of 21.62. The ablation is the more interesting comparison: the same framework with unmodified VSO reaches 44.87, so the AESS modifications account for 8.15 points, about 18% over the VSO variant. Since everything else — autoencoder, latent space, mixture, objective, hyperparameters — is held fixed between those two runs, this is the project’s most direct evidence that the adaptation mechanism and the redesigned healthy-carrier update do work the report attributes to them.

Every method in the table reports QED = 0.948. The original report suggests this may represent an effective ceiling for the task. That reading is plausible — QED is a bounded desirability score and saturates — but the supplied material contains no distributional evidence for it, so it is recorded here as the report’s hypothesis rather than as an established property of the metric.

12 Discussion

12.1 What the generation metrics say about the representation

The MAE’s combination of perfect validity, high novelty and high internal diversity is the clearest evidence that the representation layer does useful work for the rest of the system. Each metric on its own is cheap: validity is bought by SELFIES, uniqueness by any sufficiently stochastic generator, novelty by ignoring the training distribution. It is holding all four simultaneously that is hard, because the obvious ways of maximising one degrade the others.

The relevant property for DrugNet, however, is not any of the four. It is whether the decoder behaves sensibly *between* training codes, because that is where search spends its time. The latent penalty of Equation (19) and the decoder weight decay of Equation (20) are the two mechanisms intended to encourage this — keeping the code region compact and discouraging memorisation. Neither directly measures the property they are meant to induce, and the project reports no interpolation experiment, no reconstruction accuracy as a function of distance from the nearest training code, and no measure of decoder smoothness. The MOSES metrics are consistent with a well-behaved latent space, but they do not establish one, and this gap is the most substantive piece of missing evidence in the project.

12.2 The value of separating the density model

Fitting the density after training rather than during it is the design decision that most clearly distinguishes the MAE from a VAE, and the trade is a real one in both directions.

What is gained is representational freedom. The autoencoder optimises reconstruction subject only to a magnitude constraint, so the code distribution can take whatever shape reconstruction favours — multi-modal, anisotropic, non-Gaussian — rather than being pulled towards an isotropic prior. A twelve-component full-covariance mixture is then flexible enough to describe that shape after the fact, and considerably more flexible than the unit Gaussian a standard VAE would have imposed.

What is paid is coupling. The mixture is fitted once and never updated, so it approximates the code distribution of the corpus and nothing else. The moment AESS moves carriers outside that distribution — which Section 7.7 shows it does aggressively and by design — the mixture stops describing where the population is. It remains a good initialiser and a good source of replacement vectors during recovery, which is all the algorithm asks of it, but it provides no ongoing signal about whether the search has left the region where the decoder can be trusted. A system that used $p_\psi(z)$ as a soft constraint during search, rather than only as a sampler, would be the natural extension.

12.3 Adaptation as feedback control

The carrier hierarchy is a fixed multi-scale search strategy: healthy carriers cover ground, mild carriers refine towards the incumbent, severe carriers perturb locally, and the critical carrier preserves the best result. Infection moves information from stronger carriers to weaker ones and recovery prevents that transfer from eliminating diversity permanently.

Adaptation adds a feedback controller on top. Critical-carrier age is a crude stagnation statistic — it registers only whether the incumbent changed, not by how much, and not whether the population as a whole is still moving — but it is cheap, requires no additional evaluations, and is monotone in exactly the quantity a search wants to react to. The logistic response of Equation (38) then supplies hysteresis: the dead zone below $a \approx 5$

prevents the controller from reacting to the ordinary short plateaus that any stochastic search produces, which a linear response would not.

The reported gap between DrugNet with AESS and DrugNet with VSO is consistent with these mechanisms helping. It is one number from an ablation with no reported variance, so it should be read as supporting the hypothesis rather than settling it; and because adaptation and the redesigned healthy update were introduced together, the ablation cannot say which of the two is responsible.

12.4 Surrogate objectives and what they license

Replacing an expensive objective with a learned proxy is what makes the search tractable, and the substitution is standard. Its consequence is also standard: the optimiser pursues the proxy, not the thing the proxy approximates.

This is not a hypothetical concern for a population-based method. A 400-carrier population running for thirty iterations makes 12,000 evaluations per run and preferentially concentrates on whatever scores highly — including regions where DeepADTP is confidently wrong, which are precisely the regions a search is most likely to find. DeepADTP was trained on KIBA, a set of kinase-inhibitor bioactivity measurements; the 3CL protease is not a kinase, and the molecules being scored are generated rather than drawn from the training distribution. The model is therefore extrapolating in both arguments simultaneously.

None of this contradicts the reported results. It fixes what they mean. The 308 retained molecules are candidates that a particular learned function ranks highly, which is a reasonable basis for choosing what to examine next and not a claim about biological activity.

12.5 The system as a whole

The most distinctive feature of the project is the closed loop rather than any individual component. The autoencoder generates molecules but does not know the target. DeepADTP scores against the target but cannot propose molecules. AESS searches vectors but needs a meaningful space and an objective. The mixture models a density but optimises nothing. Each component supplies what its neighbour lacks, and the composition is what produces the behaviour none of them has alone.

That modularity is also the project's most useful practical property. Because the components interact only through well-defined interfaces — a latent vector, a decoded molecule, a scalar score — any one of them can be replaced without touching the others. A different scoring function substitutes directly into [Equation \(5\)](#); a different search algorithm consumes the same fitness interface, which is exactly what the VSO ablation demonstrates; a different molecular representation would require retraining but no architectural change. For a system assembled from four parts of quite different mathematical character, that is not a small thing.

13 Limitations

13.1 Computational prediction is not experimental evidence

Every result in this document is an in-silico outcome. The project contains no binding assay, no enzymatic inhibition measurement, no cellular antiviral experiment, no toxicity or pharmacokinetic assessment, and no animal or clinical work. The original report uses stronger language about molecules found “against COVID-19”; the defensible statement

is that DrugNet produced 308 candidate molecules with higher predicted fitness under its own surrogate objective than the comparison pool. The system is a candidate-generation and prioritisation engine, and its output is a list of things worth examining rather than a set of findings.

13.2 What the search dynamics imply for the Penalized LogP result

The Penalized LogP figure deserves more than a note about statistical uncertainty, because the mechanism examined in [Section 7.7](#) bears on it directly.

Two facts combine. First, the healthy-carrier update is expansive: by [Equation \(43\)](#) a healthy carrier’s latent magnitude grows geometrically, so within a few iterations of a thirty-iteration run those carriers are far outside the region the decoder was trained on, where the token distribution of [Equation \(15\)](#) carries no guarantee of chemical coherence. Second, Penalized LogP is a known-degenerate objective. Its $\log P$ term grows roughly linearly with the number of carbon atoms, its ring penalty is inactive for acyclic molecules, and its synthetic-accessibility term does not grow fast enough to compensate; the objective is therefore maximised by long unbranched aliphatic chains, which are chemically trivial and pharmaceutically useless. This is a well-understood property of the benchmark rather than a defect of DrugNet.

A search that drives its carriers far off-manifold, decoding to long repetitive token runs, is therefore well positioned to score very highly on precisely this objective. That does not invalidate the reported comparison — every method in [Table 13](#) optimises the same degenerate objective, and the VSO ablation holds everything but the search algorithm fixed, so it remains a valid statement about relative search effectiveness. But it does mean the magnitude of the number should not be read as a measure of molecular quality, and that a headline figure of 53.02 says more about how far the search travelled than about what it found. The project did not record the structures of the highest-scoring molecules, so this cannot be checked directly against the supplied material; recording them would be the single most valuable addition to a replication.

13.3 Interpretation of the MOSES comparison

The MAE was evaluated with the standard MOSES metrics against published baseline figures, but was trained on the DrugNet corpus rather than the MOSES corpus, and novelty was computed against that different training set. The results therefore establish benchmark-level performance on validity, uniqueness, novelty and diversity; they do not constitute a same-data architectural comparison in which every model saw identical training molecules.

13.4 Statistical uncertainty

The project reports means, best values and benchmark point estimates. It does not report standard deviations, confidence intervals, hypothesis tests or per-run traces. The differences can therefore be stated exactly as measured, but their stability across random initialisation, data resampling or seed choice cannot be assessed from the supplied material. This applies with particular force to the VSO ablation, which is a single-number comparison carrying most of the evidential weight for the project’s main algorithmic claim.

13.5 The representational ceiling

The project identifies its own principal limitation, and it is the right one. The decoder fixes which chemical structures can be expressed reliably, and it was trained once on a corpus assembled before the search began. AESS can move anywhere in $\mathbb{R}^{84}$, but it can only ever obtain molecules the decoder is able to produce. If a useful structural region lies outside what the decoder learned to express, no search strategy over its latent space can recover it. Everything downstream inherits this ceiling.

13.6 Reproducibility of the supplied implementation

The notebook establishes the architecture beyond reasonable doubt: recurrent widths and parameter counts, feature widths and dilation schedules, attention structure, the adaptation curve, the per-coordinate sampling in the mutation rules, the target sequence length, and the exact Penalized LogP formula. It is not, however, a runnable reproduction. It loads preprocessed arrays, pretrained weights and a serialised mixture from external paths, and several cells carry the syntax, argument-order and variable-naming inconsistencies characteristic of an edited notebook with stale outputs. [Appendix C](#) lists them individually. Train/validation/test splits, random seeds, training histories, the generated structures themselves, per-run traces and software versions are not available in the supplied files.

14 Future work

The report's own proposals point in two directions. The first is to loosen the fixed latent space, either by searching chemical space more directly through reinforcement learning or direct molecular optimisation, or by abandoning the frozen representation. The second is online learning: updating the autoencoder as the optimiser discovers new regions, so that representational capacity grows with the search rather than being fixed before it starts. Both address the ceiling described above, and the second is the more conservative of the two since it preserves the existing architecture.

Several further directions follow from the analysis in this document rather than from the original report, and are offered as such. Using the fitted mixture as a soft constraint during search, rather than only as an initialiser, would give the optimiser a signal about when it has left the region where the decoder is reliable — the missing feedback identified in [Section 12](#). Reporting decoded validity and the actual structures at every iteration would make the Penalized LogP result interpretable in the way [Section 13](#) argues it currently is not. Separately ablating adaptation and the healthy-carrier update would resolve which of the two AESS modifications carries the reported improvement. Running each configuration across multiple seeds with reported variance would give the ablation the statistical support its role in the argument requires. And measuring decoder behaviour directly — reconstruction accuracy as a function of distance from the nearest training code — would test the assumption on which the entire latent-search strategy rests.

15 Conclusion

DrugNet addresses a specific bottleneck: producing a pool of plausible molecules and ranking them against a chosen biological target quickly enough for iterative computational search. It does so by composing four components of quite different character — a

deterministic sequence autoencoder, an ex-post density model, an adaptive population-based optimiser, and an attention-based affinity predictor — into a loop that runs from latent vector to molecule to target score and back.

Within the experiments the project reports, each component performed well. The MAE reached validity 1.0000 with the highest novelty and internal diversity in its comparison table. DeepADTP reached MSE 0.121 and CI 0.890, the best figures among the listed baselines. The integrated search retained 308 candidates whose mean predicted fitness exceeded the comparison COVID-19 set by 3.6855. And on Penalized LogP, AESS reached 53.02 against 44.87 for the same framework running unmodified VSO — the project’s cleanest evidence that its algorithmic contribution does what it claims.

The defensible conclusion is that DrugNet is a coherent in-silico candidate-generation framework whose components complement one another as designed, not that it identified therapeutically effective molecules. Two qualifications carry most of the weight. The objective being optimised is a learned model, so the results describe what that model rewards rather than what binds. And the search’s healthy-carrier rule is deliberately divergent, which makes it effective at escaping the empirical latent density but also places its candidates in regions where the decoder is extrapolating — a fact that bears directly on how the largest reported number should be read.

Neither qualification undoes what was built. The architecture is sound, the components are individually well-motivated, and the modular structure means each can be replaced or ablated without redesigning the system. What a replication would most usefully add is not a new component but evidence: seeds and variance, the generated structures themselves, a direct measurement of decoder behaviour away from its training codes, and a separation of the two AESS modifications. The present record is offered so that such work can start from what was actually built rather than from what was described.

Reproducibility statement

This document is a retrospective write-up. No experiments were run for it; every number is taken from the original project report, and every architectural detail is either read from the supplied implementation or attributed to the report. Where the two disagree — and they do, in several places — both are recorded, the report’s account is followed in the scientific narrative, and the implementation’s exact behaviour is documented in [Appendix C](#). Standard derivations introduced here to explain gated recurrence, Bayesian mixtures, dilated convolution, attention and variational inference are explanatory expansions and should not be read as additional work performed by the original project.

References

- [1] Johns Hopkins Coronavirus Resource Center. COVID-19 dashboard by the center for systems science and engineering. <https://coronavirus.jhu.edu/map.html>, 2021. Figures as cited by the original project report, 25 January 2021.
- [2] Deepti Muley, Mohammad Shahin, Charitha Dias, and Muhammad Abdullah. Role of transport during outbreak of infectious diseases: Evidence from the past. *Sustainability*, 12(18):7367, 2020. doi: 10.3390/su12187367.
- [3] Amol B. Deore, Jayprabha R. Dhumane, Rushikesh Wagh, and Rahul Sonawane. The stages of drug discovery and development process. *Asian Journal of Pharmaceutical Research and Development*, 7(6):62–67, 2019. doi: 10.22270/ajprd.v7i6.616.
- [4] Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial networks. In *Advances in Neural Information Processing Systems* 27, 2014. arXiv:1406.2661.
- [5] Pierre Baldi. Autoencoders, unsupervised learning, and deep architectures. In *Proceedings of the ICML Workshop on Unsupervised and Transfer Learning*, volume 27 of *Proceedings of Machine Learning Research*, pages 37–49, 2012.
- [6] Wengong Jin, Regina Barzilay, and Tommi Jaakkola. Junction tree variational autoencoder for molecular graph generation. In *Proceedings of the 35th International Conference on Machine Learning*, 2018. arXiv:1802.04364.
- [7] Oleksii Prykhodko, Simon V. Johansson, Panagiotis-Christos Kotsias, Josep Arús-Pous, Esben Jannik Bjerrum, Ola Engkvist, and Hongming Chen. A de novo molecular generation method using latent vector based generative adversarial network. *Journal of Cheminformatics*, 11:74, 2019. doi: 10.1186/s13321-019-0397-9.
- [8] Robin Winter, Floriane Montanari, Andreas Steffen, Hans Briem, Frank Noé, and Djork-Arné Clevert. Efficient multi-objective molecular optimization in a continuous latent space. *Chemical Science*, 10(34):8016–8024, 2019. doi: 10.1039/C9SC01928F.
- [9] Sungsoo Ahn, Junsu Kim, Hankook Lee, and Jinwoo Shin. Guiding deep molecular optimization with genetic exploration. In *Advances in Neural Information Processing Systems* 33, 2020. arXiv:2007.04897.
- [10] Jiaxuan You, Bowen Liu, Rex Ying, Vijay Pande, and Jure Leskovec. Graph convolutional policy network for goal-directed molecular graph generation. In *Advances in Neural Information Processing Systems* 31, 2018. arXiv:1806.02473.
- [11] Zhenpeng Zhou, Steven Kearnes, Li Li, Richard N. Zare, and Patrick Riley. Optimization of molecules via deep reinforcement learning. *Scientific Reports*, 9:10752, 2019. doi: 10.1038/s41598-019-47148-x. arXiv:1810.08678.
- [12] Hakime Öztürk, Arzucan Özgür, and Elif Ozkirimli. DeepDTA: Deep drug–target binding affinity prediction. *Bioinformatics*, 34(17):i821–i829, 2018. doi: 10.1093/bioinformatics/bty593. arXiv:1801.10193.

- [13] Qichang Zhao, Fen Xiao, Mengyun Yang, Yaohang Li, and Jianxin Wang. AttentionDTA: prediction of drug–target binding affinity using attention model. In *2019 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*, pages 64–69, 2019. doi: 10.1109/BIBM47256.2019.8983125.
- [14] David Weininger. SMILES, a chemical language and information system. 1. introduction to methodology and encoding rules. *Journal of Chemical Information and Computer Sciences*, 28(1):31–36, 1988. doi: 10.1021/ci00057a005.
- [15] Mario Krenn, Florian Häse, AkshatKumar Nigam, Pascal Friederich, and Alán Aspuru-Guzik. Self-referencing embedded strings (SELFIES): A 100% robust molecular string representation. *Machine Learning: Science and Technology*, 1(4):045024, 2020. doi: 10.1088/2632-2153/aba947. arXiv:1905.13741.
- [16] Diederik P. Kingma and Max Welling. Auto-encoding variational Bayes. In *International Conference on Learning Representations (ICLR)*, 2014. arXiv:1312.6114.
- [17] Partha Ghosh, Mehdi S. M. Sajjadi, Antonio Vergari, Michael Black, and Bernhard Schölkopf. From variational to deterministic autoencoders. In *International Conference on Learning Representations (ICLR)*, 2020. arXiv:1903.12436.
- [18] Aäron van den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew Senior, and Koray Kavukcuoglu. WaveNet: A generative model for raw audio. In *9th ISCA Speech Synthesis Workshop*, 2016. arXiv:1609.03499.
- [19] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016. arXiv:1512.03385.
- [20] Minh-Thang Luong, Hieu Pham, and Christopher D. Manning. Effective approaches to attention-based neural machine translation. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1412–1421, 2015. arXiv:1508.04025.
- [21] Zhixin Li and Vincent Tam. A novel meta-heuristic optimization algorithm inspired by the spread of viruses. *arXiv preprint arXiv:2006.06282*, 2020.
- [22] Anna Gaulton, Anne Hersey, Michał Nowotka, A. Patrícia Bento, Jon Chambers, David Mendez, Prudence Mutowo, Francis Atkinson, Louisa J. Bellis, Elena Cibrián-Uhalte, Mark Davies, Nathan Dedman, Anneli Karlsson, María Paula Magariños, John P. Overington, George Papadatos, Ines Smit, and Andrew R. Leach. The ChEMBL database in 2017. *Nucleic Acids Research*, 45(D1):D945–D954, 2017. doi: 10.1093/nar/gkw1074.
- [23] Leslie Z. Benet, Chelsea M. Hosey, Oleg Ursu, and Tudor I. Oprea. BDDCS, the rule of 5 and drugability. *Advanced Drug Delivery Reviews*, 101:89–98, 2016. doi: 10.1016/j.addr.2016.05.007.
- [24] G. Richard Bickerton, Gaia V. Paolini, Jérémy Besnard, Sorel Muresan, and Andrew L. Hopkins. Quantifying the chemical beauty of drugs. *Nature Chemistry*, 4(2):90–98, 2012. doi: 10.1038/nchem.1243.

- [25] Jing Tang, Agnieszka Sz wajda, Sushil Shakyawar, Tao Xu, Petteri Hintsanen, Krister Wennerberg, and Tero Aittokallio. Making sense of large-scale kinase inhibitor bioactivity data sets: a comparative and integrative analysis. *Journal of Chemical Information and Modeling*, 54(3):735–743, 2014. doi: 10.1021/ci400709d.
- [26] Kyunghyun Cho, Bart van Merriënboer, Çağlar Gülçehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, 2014. arXiv:1406.1078.
- [27] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- [28] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015. arXiv:1412.6980.
- [29] Colin J. Stoneking. Bayesian inference of Gaussian mixture models with noninformative priors. *arXiv preprint arXiv:1405.4895*, 2014.
- [30] Diganta Misra. Mish: A self regularized non-monotonic activation function. *arXiv preprint arXiv:1908.08681*, 2020.
- [31] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning*, pages 448–456, 2015. arXiv:1502.03167.
- [32] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014.
- [33] Greg Landrum. RDKit: Open-source cheminformatics software. <http://www.rdkit.org/>. Used for QED, logP, synthetic accessibility and ring analysis.
- [34] Daniil Polykovskiy, Alexander Zhebrak, Benjamin Sanchez-Lengeling, Sergey Golovanov, Oktai Tatanov, Stanislav Belyaev, Rauf Kurbanov, Aleksey Artamonov, Vladimir Aladinskiy, Mark Veselov, Artur Kadurin, Simon Johansson, Hongming Chen, Sergey Nikolenko, Alán Aspuru-Guzik, and Alex Zhavoronkov. Molecular sets (MOSES): A benchmarking platform for molecular generation models. *Frontiers in Pharmacology*, 11:565644, 2020. doi: 10.3389/fphar.2020.565644. arXiv:1811.12823.
- [35] Alireza Makhzani, Jonathon Shlens, Navdeep Jaitly, Ian Goodfellow, and Brendan Frey. Adversarial autoencoders. In *International Conference on Learning Representations (ICLR) Workshop*, 2016. arXiv:1511.05644.
- [36] Tong He, Marten Heidemeyer, Fuqiang Ban, Artem Cherkasov, and Martin Ester. SimBoost: a read-across approach for predicting drug–target binding affinities using gradient boosting machines. *Journal of Cheminformatics*, 9(1):24, 2017. doi: 10.1186/s13321-017-0209-z.
- [37] Mithat Gönen and Glenn Heller. Concordance probability and discriminatory power in proportional hazards regression. *Biometrika*, 92(4):965–970, 2005. doi: 10.1093/biomet/92.4.965.

A Consolidated hyperparameters

Values marked *code* are read directly from the supplied implementation. Values marked *report* come from the project report and do not appear in the implementation, which loads pretrained weights rather than containing training calls. Values marked *default* are library defaults that the implementation does not override, and are therefore not project choices.

Table 14: Molecular Autoencoder configuration.

Item	Value	Source
Latent dimension	84	code
Sequence length	128 tokens	code
Vocabulary	85 IDs (0 = padding, 1–84 = SELFIES tokens)	code
Embedding	85×84 , masking enabled	code
Encoder recurrence	$2 \times$ bidirectional GRU, 240 units per direction	code
Encoder pooling	global max over time, $(128, 480) \rightarrow (480,)$	code
Latent projection	dense 84, linear	code
Decoder recurrence	$2 \times$ GRU, 480 units, sequences returned	code
Decoder head	time-distributed dense 85, softmax	code
Normalisation	layer normalisation after every recurrent block	code
Optimiser	Adam, $\beta_1 = 0.9$, $\beta_2 = 0.99$	code
Learning rate	5×10^{-4}	code
Gradient clipping	clip value 0.5	code
Latent penalty weight	$\frac{1}{4}$ on $\text{mean}(\mathbf{z} \odot \mathbf{z})$	code
Reconstruction scaling	$\times 128$ on the time-averaged cross-entropy	code
Declared decoder L_2	0.01 on both GRU kernels (not added to the loss)	code
Epochs / batch size	50 / 128	report
Encoder / decoder parameters	1,558,584 / 2,243,125	code

Table 15: Bayesian Gaussian mixture configuration. Items marked *default* are library settings the implementation does not override, and are therefore not project choices.

Item	Value	Source
Components K	12	code
Covariance structure	full	code
Warm start	enabled	code
Weight prior	stick-breaking (library default, not set)	default
Iteration cap	100	default
Convergence tolerance	10^{-3}	default
Covariance regularisation	10^{-6}	default
Initialisation	k -means	default
Training iterations	100	report

Table 16: DeepADTP configuration.

Item	Value	Source
Molecule / protein length	128 / 1500	code
Molecule / protein embedding	$85 \times 170 / 21 \times 170$	code
Feature width	170 channels throughout	code
Molecule / protein kernel	3 / 6	code
Dilation schedule	1, 2, 4, 8 residual; 16 final	code
Padding	causal	code
Activation	Mish	code
Attention	bidirectional unscaled dot-product	code
Dense head	850×3 , Mish	code
Dropout	0.1 after the first two dense layers	code
Output	dense 1, linear	code
Loss / optimiser / learning rate	MSE / Adam / 5×10^{-4}	report
Epochs / batch size	100 / 180	report
Parameters	3,064,421 total, 3,061,021 trainable	code

Table 17: AESS configuration.

Item	Value	Source
Population size	400 carriers	code
Iterations per run	30	report
Runs (3CL objective)	15	report
Runs (property objectives)	10	report
Initial recovery rate	0.30	code
Contact cap H_0	20	code
Mild base coefficient a	1	code
Mild attraction γ	2	code
Severe decay b	0.9	code
Infection probabilities	mild 0.20, severe 0.50, critical 0.80	code
Transition probabilities	$(0.7, 0.3)$, $(0.5, 0.5)$, $(0.3, 0.7)$	code
Adaptation coefficient	$p(a) = \sigma(a - 9)$	code
Recovery cap under adaptation	0.75	code
Healthy random vector	84-D, Uniform[0, 2)	code
Mild random vector	84-D, Uniform[0, 1)	code
Severe base intensity	$s_0 = 1/U$, $U \sim \text{Uniform}(0, 1)$	code
Mild base intensity	$m_0 = 0.1$ (see Appendix C)	code
Retained carriers	severe and critical only	code

Verification of the reported parameter counts

The parameter counts above are taken from model summaries stored in the notebook. Those summaries are stale outputs and are therefore not self-validating, so they were recomputed analytically from the layer specifications. All five reproduce exactly.

For the recurrent layers the relevant formula is the TensorFlow 2 default `reset_after=True` variant, in which each of the three gate blocks carries an input kernel, a recurrent kernel, and *two* bias vectors:

$$P_{\text{GRU}} = 3(d_{\text{in}}u + u^2 + 2u), \quad (57)$$

for input width d_{in} and u units, doubled for a bidirectional layer. A 1-D convolution contributes $Kd_{\text{in}}d_{\text{out}} + d_{\text{out}}$, a dense layer $d_{\text{in}}d_{\text{out}} + d_{\text{out}}$, layer normalisation $2d$, and batch normalisation $2d$ trainable plus $2d$ non-trainable.

Applying these gives $7,140 + 469,440 + 960 + 1,039,680 + 960 + 40,404 = 1,558,584$ for the encoder and $815,040 + 960 + 1,385,280 + 960 + 40,885 = 2,243,125$ for the decoder. For DeepADTP, five protein convolutions at 174,250 each (including batch-normalisation state), five molecule convolutions at 87,550, two embeddings and the dense head sum to 3,064,421, of which the ten batch-normalisation layers account for the 3,400 non-trainable parameters, leaving 3,061,021 trainable.

That the attention modules contribute nothing confirms independently that the implementation uses the unscaled dot-product form of Equation (48) rather than a scaled or additive variant, since a learned scale would have added parameters the stored totals do not contain. The script `supplementary/verify_parameter_counts.py` reproduces this check.

B Bayesian Gaussian mixture: variational updates

This appendix completes the derivation sketched in Section 6. Everything here is standard Bayesian mixture theory; the DrugNet-specific facts are only $K = 12$, full covariance matrices, and that the fitted model is used purely as a sampler. Prior hyperparameters are library defaults rather than project settings, so the derivation is given in general conjugate form.

B.1 Complete-data likelihood

With one-hot assignment indicators $c_{nk} = \mathbb{1}(c_n = k)$, the joint density of codes and assignments given the parameters is

$$p(Z, C \mid \pi, \mu, \Sigma) = \prod_{n=1}^N \prod_{k=1}^K [\pi_k \mathcal{N}(z_n \mid \mu_k, \Sigma_k)]^{c_{nk}}. \quad (58)$$

Taking logarithms turns the products into sums and the exponents into weights, which is what makes the expectation step tractable: given the responsibilities $\mathbb{E}[c_{nk}] = r_{nk}$, the expected complete-data log-likelihood is linear in those weights.

For a point-estimate mixture the responsibilities are

$$r_{nk} = p(c_n = k \mid z_n) = \frac{\pi_k \mathcal{N}(z_n \mid \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(z_n \mid \mu_j, \Sigma_j)}, \quad (59)$$

a softmax over components of the log joint density. Equation (28) is the Bayesian analogue, in which the parameters are replaced by expectations under the variational posterior.

B.2 Variational factorisation and the updates

The posterior is approximated by the factorisation

$$q(C, \pi, \mu, \Lambda) = q(C) q(\pi) \prod_{k=1}^K q(\mu_k, \Lambda_k), \quad (60)$$

and the ELBO of Equation (27) is maximised by coordinate ascent. Under the conjugate priors of Equation (26) each factor retains its prior's form, so the updates are updates of

hyperparameters. Writing N_k , $\bar{z}_k$ and S_k for the weighted statistics of Equation (29), the parameter factors update as

$$\alpha_k = \alpha_0 + N_k, \quad (61)$$

$$\beta_k = \beta_0 + N_k, \quad (62)$$

$$\mathbf{m}_k = \frac{1}{\beta_k} (\beta_0 \mathbf{m}_0 + N_k \bar{z}_k), \quad (63)$$

$$\nu_k = \nu_0 + N_k, \quad (64)$$

$$\mathbf{W}_k^{-1} = \mathbf{W}_0^{-1} + N_k S_k + \frac{\beta_0 N_k}{\beta_0 + N_k} (\bar{z}_k - \mathbf{m}_0) (\bar{z}_k - \mathbf{m}_0)^\top. \quad (65)$$

The expectations needed by Equation (28) follow in closed form,

$$\mathbb{E}_q[\log \pi_k] = \psi(\alpha_k) - \psi\left(\sum_j \alpha_j\right), \quad (66)$$

$$\mathbb{E}_q[\log |\Lambda_k|] = \sum_{i=1}^d \psi\left(\frac{\nu_k + 1 - i}{2}\right) + d \log 2 + \log |\mathbf{W}_k|, \quad (67)$$

with ψ the digamma function, and

$$\mathbb{E}_q\left[(z_n - \mu_k)^\top \Lambda_k (z_n - \mu_k)\right] = \frac{d}{\beta_k} + \nu_k (z_n - \mathbf{m}_k)^\top \mathbf{W}_k (z_n - \mathbf{m}_k). \quad (68)$$

Alternating Equation (28) with Equations (61) to (65) increases the ELBO monotonically and converges to a local optimum.

B.3 Effective components and posterior sampling

The Dirichlet update Equation (61) is what produces automatic model selection. A component that no code claims has $N_k \approx 0$, so $\alpha_k \rightarrow \alpha_0$; with a small concentration prior its expected weight is driven towards zero and it contributes negligibly. The number of *effective* components can therefore be smaller than K , which is why $K = 12$ functions as an upper bound rather than an assertion that the latent cloud has exactly twelve modes.

Sampling uses the fitted posterior means, as in Equation (30): draw a component from the fitted weights, then draw from that component's Gaussian. A fully Bayesian posterior predictive would marginalise the parameter uncertainty, giving a multivariate Student- t per component rather than a Gaussian. The library's sampling method uses the plug-in Gaussian form, and that is what DrugNet uses; with $N \approx 1.2$ million codes the two are numerically indistinguishable, but the distinction is recorded so that the description matches what runs.

C Notebook audit and reproducibility caveats

The supplied notebook establishes the architecture but is not an executable reproduction. Several cells carry defects characteristic of an edited notebook retaining outputs from earlier versions of its source. They are listed here individually so that a replication can distinguish the scientific design from implementation accidents, and so that nothing in the main text has to be quietly adjusted to make the account consistent.

Three of these prevent execution outright. Two more are silent and change what the algorithm does.

C.1 Defects that prevent execution

Fitness dispatch (syntax). The fitness function's first branch is written `if typ="3CL":`, an assignment inside a condition, and the second branch `elif typ=="QED"` omits its colon. The cell does not parse.

Adaptation and reset (syntax). Both methods are written `if self.VSO==False` without a colon, and both use `continue` in an `else` branch outside any loop. Two further parse errors.

Output variable names. The export cell allocates `generated_solutions_1`, writes to `generated_solutions`, and the save cell references `generated_sols`. Three names for one array.

C.2 Defects that change behaviour silently

Mild base intensity. The carrier constructor sets `m_intensity = solution.shape[0]/10`. Mixture samples are returned with shape `(1,84)`, so `shape[0]` is 1 and the base intensity is 0.1 for every carrier, rather than the $84/10 = 8.4$ the expression appears to intend. The constant term of the mild update Equation (34) is consequently two orders of magnitude smaller than the attraction term for typical separations, so mild carriers are driven almost entirely by attraction to the incumbent.

Adaptation decay cast. The severe decay is updated as `decay = min(int(init_decay + p/10), 0.99)`. With $b_0 = 0.9$ and $p \in [0,1)$ the argument of `int` lies in $[0.9, 1.0)$, so the cast yields 0 and the decay becomes 0. This sets $\eta = 0$ in Equation (35), freezing severe carriers completely from the first call to adaptation onwards. The report describes a gradual *increase* in this coefficient, so the intended expression is almost certainly `min(init_decay + p/10, 0.99)` without the cast. The main text follows the report; this is what the code does.

Mutable infection-rate aliasing. The constructor binds `self.init_inf_rates = infection_rates`, the same dictionary object rather than a copy. Adaptation then divides the rates in place by Equation (42), which mutates the object that `reset()` restores from. The infection probabilities therefore decay monotonically across a run and are never actually reset when a new critical carrier is found.

Shared array references on infection. The severe infection path assigns `healthy[j].solution = infected[i].solution` by reference. Subsequent in-place `+=` updates in mutation then modify both carriers simultaneously, aliasing them permanently. A `.copy()` is required.

Missing fitness argument. Calls to the fitness function inside `infect()` and `recover()` pass only the solution, omitting the objective-type argument the definition requires.

Constructor argument alignment. The search object is instantiated with eleven positional arguments against a signature whose final two parameters are `fitness_type="3CL"` and `VSO=False`. The trailing `False` therefore binds to `fitness_type`, not `VSO`, and the

fitness dispatch matches no branch. A working invocation needs `fitness_type="3CL"` passed by keyword.

Stale-list mutation during infection. `make_lists()` is called inside the inner infection loop, rebuilding the healthy and infected dictionaries while the sorted lists derived from them are still being iterated and indexed.

Recovery can consume the incumbent. Recovery selects from the infected set, which includes the critical carrier, and replaces solutions before checking class. The elite solution can therefore be resampled away without being demoted, defeating the elitism that [Section 7](#) otherwise enforces.

Unapplied decoder regularisation. As set out in [Section 5](#), both decoder GRUs declare an L_2 kernel regulariser but the custom training step never adds the layer regularisation losses to the objective, so [Equation \(20\)](#) does not contribute to the gradient.

Masking through convolutions. Both DeepADTP embeddings enable masking, but the convolutional and normalisation layers that follow do not consume Keras masks, so padded positions participate in the convolutions and the batch statistics. For the 3CL target this affects 1194 of 1500 protein positions.

C.3 Cosmetic evidence of editing

The stored model summaries name the models `rae_encoder` and `rae_decoder` while the construction function assigns the names `mae_encoder` and `mae_decoder`. The outputs predate a rename from RAE (regularised autoencoder) to MAE. This has no functional consequence, but it corroborates the deliberate deterministic-regularised-autoencoder framing described in [Section 2](#), and it is a reminder that the stored outputs were produced by an earlier version of the source.

C.4 Recommended replication policy

A clean reproduction should: pin exact package versions; convert the notebook cells into importable modules; pass every constructor argument by keyword; copy arrays at every carrier assignment; unit-test each AESS transition against the equations in [Section 7](#); add the regularisation losses to the autoencoder objective if the L_2 term is intended; apply explicit masking in DeepADTP; record random seeds and data splits; save per-iteration carrier states, decoded molecules and validity rates; and re-run every benchmark from a fresh kernel with reported variance across seeds.

D Source fidelity

This document was reconstructed from two artefacts: the original project’s implementation notebook, and a prior English rendering of the original Turkish project report. The Turkish original was not available during preparation, so statements attributed here to “the report” are carried forward from that rendering and could not be checked against the primary document.

Numerical results, benchmark tables, data-processing decisions, component names and reported hyperparameters are preserved from the source material without alteration.

Architecture details are described as code-verified only where they appear unambiguously in the notebook, and [Tables 14 to 17](#) mark the provenance of every value. Standard mathematics introduced to explain gated recurrence, Bayesian mixtures, variational inference, dilated convolution and attention is explanatory expansion and is not a claim about additional work performed by the project.

Where the report describes an intended method and the notebook as written does something different — by syntax error, integer cast, aliasing or omission — the main narrative follows the report’s scientific intent and [Appendix C](#) records the code-level behaviour separately. Two observations in this document go beyond both sources: the analysis of the healthy-carrier update’s expansive character in [Section 7.7](#), and its implication for the Penalized LogP result in [Section 13](#). Both follow directly from the update equation as implemented and from published properties of the benchmark objective, and both are identified as analysis rather than as findings of the original project.

The supplementary material contains the two checks the text refers to: an analytic recomputation of every reported parameter count, and a corrected reference implementation of the search operators whose self-test verifies the elitism, conservation and divergence properties claimed in [Section 7](#). Neither reproduces a DrugNet experiment, and neither is offered as one.